arXiv:2410.08049v1 [cs.CV] 10 Oct 2024

Scaling Up Your Kernels: Large Kernel Design in ConvNets towards Universal Representations

Yiyuan Zhang, Xiaohan Ding, Xiangyu Yue

Abstract—This paper proposes the paradigm of large convolutional kernels in designing modern Convolutional Neural Networks (ConvNets). We establish that employing a few large kernels, instead of stacking multiple smaller ones, can be a superior design strategy. Our work introduces a set of architecture design guidelines for large-kernel ConvNets that optimize their efficiency and performance. We propose the UniRepLKNet architecture, which offers systematical architecture design principles specifically crafted for large-kernel ConvNets, emphasizing their unique ability to capture extensive spatial information without deep layer stacking. This results in a model that not only surpasses its predecessors with an ImageNet accuracy of 88.0%, an ADE20K mIoU of 55.6%, and a COCO box AP of 56.4% but also demonstrates impressive scalability and performance on various modalities such as time-series forecasting, audio, point cloud, and video recognition. These results indicate the universal modeling abilities of large-kernel ConvNets with faster inference speed compared with vision transformers. Our findings reveal that large-kernel ConvNets possess larger effective receptive fields and a higher shape bias, moving away from the texture bias typical of smaller-kernel CNNs. All codes and models are publicly available at https://github.com/AILab-CVC/UniRepLKNet, promoting further research and development in the community.

Index Terms—Convolutional Neural Network, Large-kernel ConvNets, Multimodal Learning, Neural Network Architecture Design

1 Introduction

Convolutional neural networks (ConvNets) are widely adopted in the computer vision community [3], [4], [5], [6], [7], [8], [9], [10], [11], [12], [13], [14], [15], [16], [17], [18]. Recently, the dominance of ConvNets has been significantly challenged by Vision Transformers (ViTs) [19], [20], [21], [22], [23] which utilize global attention [19], [22], [24] and window-based attention [20], [25], [26]. In addition to image recognition, ViTs are also widely applied across various modalities [27], [28], [29], including audio [30], point cloud [31], video [32], etc., demonstrating their potent capability of universal modeling for perception tasks. However, the quadratic complexity, high memory costs, and slow inference speed hinder broader applications of ViTs, such as the perception of high-resolution images and long-form videos. Therefore, we ask the following question:

*Can we build a **ConvNet** that offers similar **universal modeling** capabilities as ViT, but with reduced complexity and significantly faster inference speed?*

Diving into the advantages of ViTs, the global attention mechanism brings out long-range dependencies and contextual relationships [33], [34], [35], [36], [37]. This prompts us to consider: *how to enhance long-range dependencies and contextual relationship in ConvNets?* Large convolutional kernels appear to be the solution for ConvNets after a decade's exploration [1], [2], [16], [38], [39], [40]. In 2014, Xu *et al*. [38] proposed the inverse kernel and deconvolution to add larger spatial support for image denoising. Following this, large kernels were introduced to segmentation tasks in 2017 for larger ERFs [39]. Additionally, in 2022, Liu *et al*. [16] scaled kernels up to 7×7 within the macro architecture

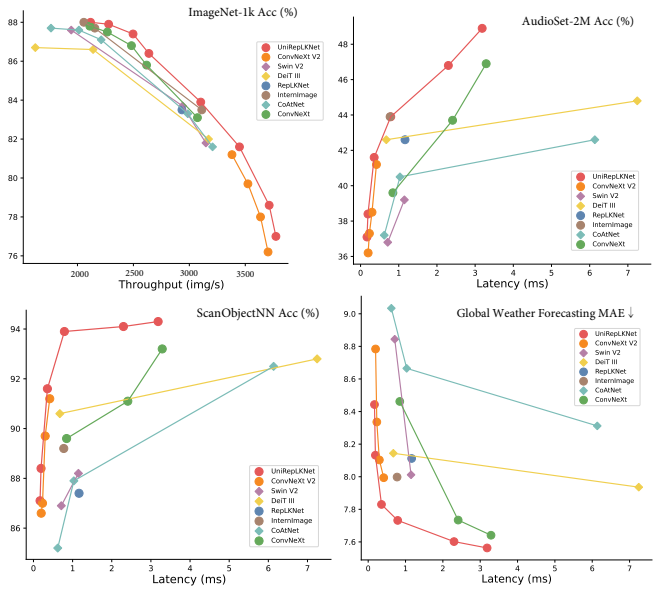

Fig. 1: **UniRepLKNet models learn universal representation across multiple modalities**. Regarding precision and efficiency across image, audio, point Cloud, and time-series modalities, UniRepLKNet delivers better scaling abilities between performance and computation burdens. The latency is tested with an A100 GPU, batch size of 128, and full precision (fp32).

- *Y. Zhang is with the Department of Information Engineering, The Chinese University of Hong Kong, Hong Kong, China. (Email: yiyuanzhang.ai@gmail.com)*
- *X. Ding is with Tencent AI Lab, Shenzhen, China.*
- *X. Yue is with the Department of Information Engineering, The Chinese University of Hong Kong, Hong Kong, China. (Email:xyyue@ie.cuhk.edu.hk)*
- *Preliminary versions of this work have appeared in CVPR 2022 [1] and CVPR 2024 [2].*

TABLE 1: Inference speed of a stack of 24-layer depth-wise convolutions with various kernel sizes and resolutions on a single GTX 2080Ti GPU. The input shape is (64, 384, R, R). Baselines are evaluated with Pytorch 1.9.0 + cuDNN 7.6.5, in FP32 precision.

Resolution R	Impl.	Latency (ms) @ Kernel size									
		3	5	7	9	13	17	21	27	29	31
16×16	Pytorch	5.6	11.0	14.4	17.6	36.0	57.2	83.4	133.5	150.7	171.4
	Ours	5.6	6.5	6.4	6.9	7.5	8.4	8.4	8.4	8.3	8.4
32×32	Pytorch	21.9	34.1	54.8	76.1	141.2	230.5	342.3	557.8	638.6	734.8
	Ours	21.9	28.7	34.6	40.6	52.5	64.5	73.9	87.9	92.7	96.7
64×64	Pytorch	69.6	141.2	228.6	319.8	600.0	977.7	1454.4	2371.1	2698.4	3090.4
	Ours	69.6	112.6	130.7	152.6	199.7	251.5	301.0	378.2	406.0	431.7

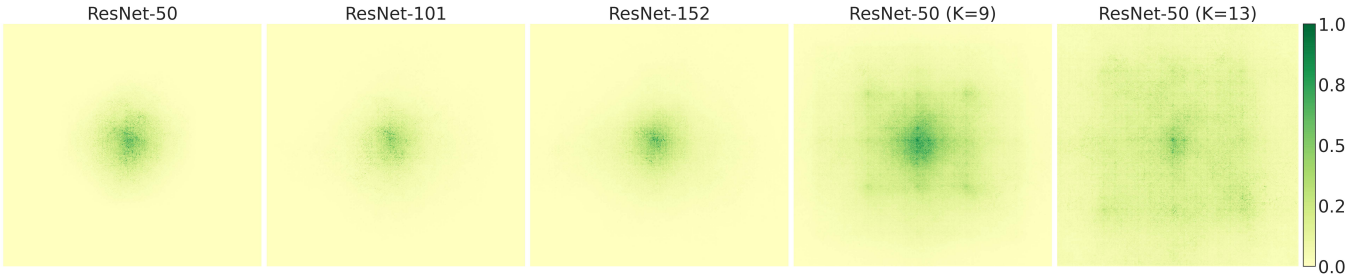

Fig. 2: The *Effective Receptive Field (ERF)* of ResNet-50/101/152 and the large kernel (K) variants of ResNets, respectively. A more widely distributed dark area indicates a larger ERF. More layers (*e.g.*, from ResNet-101 to ResNet-152) help little in enlarging ERFs. Instead, the large-kernel ConvNets effectively obtain large ERFs.

of Swin Transformer [41]. Then SLak [40] utilized sparse large kernels of size 51×51, demonstrating the efficiency and superiority of large-kernel ConvNets. Despite these advancements, a significant question becomes more clear: *How can we design a large-kernel ConvNet with universal modeling abilities, high efficiency, and promising scalability for both data and parameters?*

In this paper, we explore the design of an efficient and universal architecture, specifically large-kernel ConvNets, by rethinking the traditional design of using a deep stack of small kernels. When we add a 3×3 convolution to a small-kernel ConvNet, we expect it to have three simultaneous effects - **1)** expanding the receptive field, **2)** increasing the abstraction hierarchy of spatial patterns (*e.g.*, from angles and textures to object shapes), and **3)** improving the model's general representation capability by increasing its depth, thus introducing more learnable parameters and non-linearities. In contrast, we argue that such three effects in a large-kernel architecture should be decoupled, as the model should leverage the substantial strength of large kernels - *the ability to see wide without going deep*. Since increasing the kernel size is more effective than stacking layers for enlarging the ERF [42] [1], a sufficient ERF can be established with only a few large-kernel layers. This allows the compute budget to be allocated to other efficient structures that more effectively increase the abstract hierarchy of spatial patterns or the overall depth. For example, when the objective is to extract higher-level local spatial patterns from lower-level ones, a 3×3 convolution might be a more suitable option than a large-kernel convolution layer. The reason is that the latter demands more computations and may result in patterns that are no longer confined to smaller local regions, which could be undesirable in specific scenarios.

Concretely, we propose a roadmap (§ 3) to *Uniervsal ConvNets* on both macro and micro designs of a large-kernel ConvNet architecture:

- **Step 1**: making large-kernels practical (§ 3.1), which should be both efficient (§ 3.1.1) and effective (§ 3.1.2).
- **Step 2**: designing a modern large-kernel ConvNet architecture, including deep blocks design (§ 3.2.1), micro design with structural re-paramterization (§ 3.2.2), kernel size principle (§ 3.2.3), and scaling rules (§ 3.2.4) of large kernel ConvNets, respectively.
- **Step 3**: generalizing large-kernel ConvNets to multiple modalities including time-series, audio, point cloud, and video (§ 3.3).
- **Step 4**: fusing multimodal features with large kernel convolution operators, an alternative to the cross-attention mechanism (§ 3.4).

A ConvNet constructed following such guidelines (Fig. 3) achieves the three aforementioned effects separately. It utilizes a modest number of large kernels to guarantee a large ERF, as shown in Fig. 2, employs small kernels to extract complicated spatial patterns more efficiently, and incorporates multiple lightweight blocks to further increase depth and enhance representational capacity.

As shown in Fig. 1, our architecture achieves leading performance on universal understanding tasks including ImageNet classification [43], AudioSet-2M [44], ScanObjectNN [45], and Global Weather Forecasting tasks [46]. In image recognition, UniRepLKNet outperforms existing large-kernel ConvNets such as RepLKNet [1], SLaK [40], and recent powerful architectures including ConvNeXt V2 [47], FastViT [48], Swin V2 [49] and DeiT III [50], in terms of both accuracy and efficiency. Moreover, our ar-

1. Referring to this paper, the growth order of ERF is $\mathcal{O}(k\sqrt{n})$, where k is the kernel size and n is the depth of the convolutional layer.

chitecture exhibits a significantly higher shape bias [51], [52] compared to existing ConvNets and ViTs. Specifically, it makes predictions based more on the overall shapes of objects than on textures, which aligns with the human visual system and results in better generalization. This may explain its superiority in downstream tasks. In addition, as we scale our model to 1.4B with training data of 10B image-text pairs from LAION-5B dataset [53] for CLIP [54] pretraining, it demonstrates impressive zero-shot abilities across 26 datasets (Table 13) on the widely adopted CLIP benchmark[2]. Moreover, UniRepLKNet also shows outstanding performance on the large vision-language model benchmarks (Table 14).

RepLKNet [1] was proposed partly "in defense of ConvNets" as ViTs began to dominate multiple image recognition tasks previously led by ConvNets. Moreover, given that transformers have demonstrated universal perception capability across multiple modalities [28], [55], this work aims not only to reclaim the leading position in image recognition tasks by surpassing the performance of ViTs but also to contribute to areas where ConvNets were not traditionally dominant. Specifically, we achieve impressive performance on *audio, video, point cloud, and time-series* tasks, with remarkably universal and simple solutions. We use modality-specific preprocessing approaches to transform all data into 3D embedding maps, similar to how images are processed, and use the same architecture as the backbone to process these embedding maps. Our model demonstrates ***uni*versal perception ability across multiple modalities with a *uni*fied architecture**, hence the name **UniRepLKNet**. Impressively, UniRepLKNet achieves remarkable results even on modalities that were not considered the stronghold of ConvNet, *e.g.*, audio and temporal data. On a large-scale time-series forecasting task predicting the global temperature and wind speed, UniRepLKNet even outperforms the latest state-of-the-art transformer customized for the task. These results not only signify a "***comeback***" for ConvNet in its original domain but also highlight the potential of large-kernel ConvNet to "***conquer***" new territories, expanding its applicability and versatility across various tasks.

This work builds upon our preliminary conference papers in CVPR 2022 [1] and CVPR 2024 [2], and we present a substantial extension of it in various aspects. *First*, we further develop the large-kernel convolution operators as a higher-efficiency alternative of attention mechanism on both learning universal representations (§ 3.1 & § 3.2 & § 3.3) and fusing diverse features across modalities (§ 3.4). *Second*, we continue to explore the potential of large-kernel ConvNets on additional large-scale multimodal comprehension abilities (Table 8 & Table 9 & Table 10) including AudioSet-2M for audio and Objaverse for point clouds, *etc*. *Third*, we scale the proposed architectures to 1.4B parameters and validate the transferable abilities of UniRepLKNet in learning 10 billion image-text pairs with CLIP [56] for zero-shot recognition tasks, further illustrating their efficiency and advancements in architectural and data scalability (Table 13). *Fourth*, to thoroughly investigate the efficiency advantages of ConvNets, we use UniRepLKNet for training large vision-language models (Table 14), which shows promising performance on comprehensive zero-shot visual question-answering benchmarks. *Last but not least*, we summarize the architectural design as a roadmap to universal ConvNets, hoping to foster research efforts in designing more efficient architectures.

2. https://github.com/LAION-AI/CLIP_benchmark

2 Related Works

Large kernels in early ConvNets. Early ConvNets, such as AlexNet [3] and Inception [4], [5], [6], initially used large kernels (7×7 or 11×11) to capture spatial features. However, the trend shifted with VGG-Net, which favored smaller, more frequent layers [18]. Innovatively, the Global Convolution Network (GCN) [39] utilized very large kernels ($1 \times K$ followed by $K \times 1$) to improve semantic segmentation. Local Relation Networks (LR-Net) [57] explored dynamic kernel sizes and found that performance peaked with 7×7 kernels but declined with larger sizes, illustrating the challenges of balancing kernel size with network efficiency.

Explorations with large kernels. Expanding the traditional definition of kernels in convolutional networks, Swin Transformer [20] innovatively employed shifted attention mechanisms with window sizes ranging from 7 to 12, effectively functioning as dynamic kernels. Research by Han *et al.* [35] demonstrated that replacing the attention layers in Swin Transform with either static or dynamic 7×7 convolution layers yielded results comparable to the original model. Additionally, the MetaFormer [58] proposed that a large-kernel pooling layer could serve as a viable alternative to self-attention mechanisms. Further extending the concept, the Global Filter Network (GFNet) [59] refined spatial connection weights via the Fourier domain, achieving a global convolution effect similar to circular convolutions in the spatial domain, underscoring the versatile applications of large-scale kernels across different network architectures.

Modern ConvNets with very large kernels. The introduction of RepLKNet [1] marked a significant shift in ConvNet design by demonstrating that enlarging kernel sizes can improve performance, particularly in downstream applications. This approach introduced several key design strategies, such as integrating shortcuts with large kernels for better microstructural efficiency. While RepLKNet was inspired by the straightforward architecture of the Swin Transformer, subsequent research has expanded on this idea. Liu *et al.* [40] and others pushed the boundaries further by scaling up kernel sizes, applying these concepts to 3D vision tasks [60], image dehazing [61] and super-resolution [62]. Despite these advances, the architectural nuances of ConvNets with large kernels remain relatively unexplored, indicating a promising area for future research.

The growing interest in large-kernel ConvNets is driven by their effectiveness in capturing fine-grained and global spatial features. However, existing models often integrate large kernels with additional mechanisms, limiting the understanding of their standalone potential. Research shows that scaling kernel sizes improves performance, yet a universal large-kernel ConvNet architecture remains undeveloped. This work proposes a simplified, universal design that retains the spatial extraction benefits of large kernels, bridging the flexibility of Transformer models with the efficiency

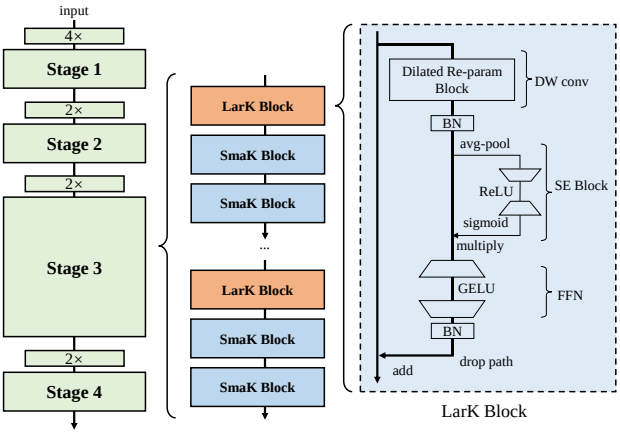

Fig. 3: Architectural design of UniRepLKNet. A LarK Block comprises a Dilated Reparam Block proposed in this paper, an SE Block [63], an FFN, and Batch Normalization (BN) [64] layers. The only difference between a SmaK Block and a LarK Block is that the former uses a depth-wise 3×3 conv layer in replacement of the Dilated Reparam Block in the latter. Stages are connected by down-sampling blocks implemented by stride-2 dense 3×3 conv layers. We may flexibly arrange the blocks in different stages and the details of our provided instances are shown in Table 4.

of traditional ConvNets, and extending applicability across diverse tasks.

3 A Roadmap to Universal ConvNets

Our roadmap to universal large-kernel ConvNets (UniRepLKNet, Fig. 3) comprises four steps: **1**) We first explore why large kernel convolutions are not commonly used in modern ConvNets and propose 5 guidelines to make them more practical and evaluate their effectiveness (§ 3.1). **2**) We propose 4 guidelines for building a powerful and competitive large-kernel ConvNet architecture (§ 3.2). **3**) We propose to generalize the large-kernel ConvNets to multimodal understanding tasks (§ 3.3). **4**) Finally, we propose asymmetric large-kernel convolution to efficiently fuse multimodal features in contrast to cross-attention (§ 3.4).

3.1 Step 1: Making Large Kernels Practical

3.1.1 Making Large Kernels Efficient

The first reason why large kernels were rarely used is that they were believed to be computationally expensive due to the quadratic increase in the number of parameters and FLOPs with kernel size. However, we argue that this drawback can be significantly mitigated by using depth-wise (DW) convolutions [14], [17]. As DW convolutions only consume a minor fraction of the total computational budget of a ConvNet, increasing the kernel sizes does not significantly make the model larger or slower. For example, as shown in Table 2c, increasing the kernel sizes of DW convolutions in MobileNet V2 [65] from 3×3 to 13×13 results in only a 2.7% increase in FLOPs and 4.2% increase in parameters, which is acceptable given the corresponding +2.31% mIoU improvement in Cityscapes segmentation. The remaining 1×1 convolutions dominate most of the complexity.

One may be concerned that DW convolutions could be inefficient on modern parallel computing devices, such as GPUs. It is true for conventional DW 3×3 kernels [11], [17], [65], as DW operations introduce a low ratio of computation *vs.* memory access cost [66], which is not friendly to modern computing architectures. However, we find that as the kernel size increases, the computational density also increases. For example, in a DW 11×11 kernel, each value loaded from the feature map can be used in up to 121 multiplications, while in a 3×3 kernel, the number is only 9. Therefore, according to the roofline model [65], the actual latency should not increase as much as the FLOPs when the kernel size becomes larger.

The discussions above reveal that large-kernel DW convolutions can run faster with better implementation. In practice, we propose a block-wise (inverse) *implicit GEMM* algorithm to replace the original operator. [3] Table 1 shows that our implementation is significantly more efficient compared to the PyTorch baseline.

Therefore, we propose our first guideline as follows. **Guideline 1: use depth-wise large-kernel convolution with proper operator-level implementation.**

3.1.2 Making Large kernels Effective

The second reason why large kernels were rarely used is that they were believed to harm the model's performance. However, we argue that large kernels are not harmful; they were simply not used properly. We propose three guidelines to use large kernels correctly in modern ConvNets.

Guideline 2: identity shortcut is vital, especially for networks with very large kernels. To demonstrate this, we use *MobileNet V2* [65] for benchmarking, since it heavily employs DW layers and has two published variants (with or without shortcuts). For the large-kernel counterparts, we simply replace all the DW 3×3 kernels with 13×13. All the models are trained on ImageNet with identical training configurations for 100 epochs (see Appendix A for details). Table 2a shows that large kernels improve the accuracy of MobileNet V2 with shortcuts from 71.76% to 72.53%. However, for the model without shortcuts, large kernels reduce the accuracy to only 53.98%. We explain this phenomenon from a perspective similar to [67]: shortcuts make the model an implicit ensemble of numerous models with different receptive fields (RFs), allowing it to benefit from a much larger maximum RF without losing the ability to capture small-scale patterns.

Guideline 3: re-parameterizing large kernels with small kernels improves the performance. To better understand the effect of the aforementioned ensemble of different RFs, we explore whether using small kernels to produce a bigger ensemble of more different RFs improves the performance. Specifically, we replace the 3×3 layers of *MobileNet V2* with 9×9 and 13×13, and optionally adopt the *Structural Re-parameterization* [12], [68], [69] methodology to add small kernels without altering the inference structure of the resultant model. Specifically, we construct a 3×3 layer parallel

3. For PyTorch, we have released the efficient implementation at https://github.com/AILab-CVC/UniRepLKNet as a plug-and-play module.

TABLE 2: **Architectural design choices of Step 1: Making Large Kernels Practical**. We report the Top-1 Accuracy (%) on the ImageNet-1k classification and mIoU (%) for Cityscapes, and ADE-20K segmentation tasks.

(a) Kernel sizes & shortcut of MobileNet V2.

Shortcut	Kernel size	IN-1k (%)
	3×3	**68.67**
✓	3×3	**71.76** ↑ 3.09
	13×13	53.98
✓	13×13	**72.53** ↑ 18.55

(b) Kernel sizes & re-parameterization on MobileNet V2.

Kernel size	3×3 re-param	IN-1k (%)	Cityscapes (%)
9×9		72.67	76.11
9×9	✓	**73.09** ↑ 0.42	**76.30** ↑ 0.19
13×13		72.53	75.67
13×13	✓	**73.24** ↑ 0.71	**76.60** ↑ 0.93

(c) Kernel sizes in the *last stage* of MobileNet V2.

Kernel size	IN-1k (%)	Cityscapes (%)	#Params	FLOPs
3×3	71.76	72.31	2.64M	214.5M
7×7	72.00	74.30	2.67M	215.9M
9×9	71.83	74.15	2.69M	217.1M
13×13	**71.97** ↑ 0.21	**74.62** ↑ 2.31	2.75M (+4.2%)	220.2M(+2.7%)

(d) Kernel sizes of different stages applying (a) (b) (c) in base model.

S1-S2-S3-S4	IN-1k Classification			ADE20K Segmentation		
	Top-1 (%)	Params	FLOPs	mIoU (%)	Params	FLOPs
3-3-3-3	82.11	71.8M	12.9G	46.05	104.1M	1119G
7-7-7-7	82.73	72.2M	13.1G	48.05	104.6M	1123G
13-13-13-13	83.02	73.7M	13.4G	48.35	106.0M	1130G

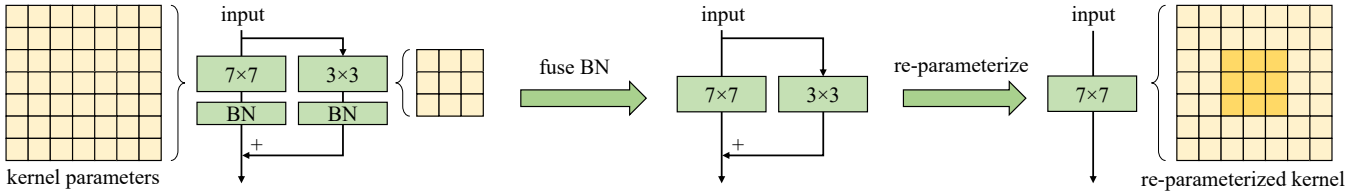

Fig. 4: An example of re-parameterizing a small kernel (*e.g.*, 3×3) in Table 2b into a large one (*e.g.*, 7×7). We use the structural re-parameterization as previous practices [12], [68].

to the large-kernel layer and add their outputs together after the *Batch normalization (BN)* [64] layers (Fig. 4). After training, we merge the small kernel and BN parameters into the large kernel, so the resultant model will be mathematically equivalent to the training model but no longer has small kernels. Table 2b shows that directly increasing the kernel size from 9 to 13 reduces accuracy, while re-parameterization addresses this issue.

We then transfer the ImageNet-trained models to semantic segmentation with DeepLabv3+ [70] on Cityscapes [71]. We only replace the backbone and keep all the default training settings of MMSegmentation [72]. The observation is similar to that on ImageNet: 3×3 re-parameterization improves the mIoU of the 9×9 model by +0.19% and the 13×13 model by +0.93%; with re-parameterization, increasing the kernel size from 9 to 13 no longer degrades performance on either ImageNet or Cityscapes.

Guideline 4: large kernels (*e.g.*, 13×13) are effective even on small feature maps (*e.g.*, 7×7). To validate it, We enlarge the DW convolutions in the *last stage* of *MobileNet V2* to 7×7 or 13×13, hence the kernel size is on par with or even larger than feature map size (7×7 by default). We apply re-parameterization to the large kernels as suggested by Guideline 3. Table 2c shows although convolutions in the last stage already involve very large receptive field, further increasing the kernel sizes still leads to performance improvements, especially on downstream tasks such as *Cityscapes*.

Remark. When kernel size becomes large, notice that the translational equivariance of CNNs does not strictly hold. As illustrated in Fig. 5, two outputs at adjacent spatial locations share only a fraction of the kernel weights, *i.e.*, and are transformed by different mappings. The property also agrees with the "philosophy" of *ViTs* – relaxing the symmetric prior to obtaining more capacity. Interestingly, we find 2D *Relative Position Embedding (RPE)* [73], [74], which is widely used in the transformer community, can also be viewed as a large depth-wise kernel of size

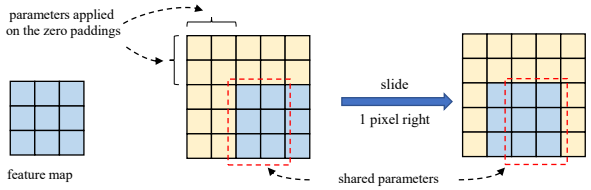

Fig. 5: Illustration to convolution with small feature map and large kernel. Two outputs at adjacent locations only share a part of kernel weights. Translational equivariance does not strictly hold.

$(2H-1)\times(2W-1)$, where H and W are feature map height and width respectively. Large kernels not only help to learn the relative positions between concepts but also encode the *absolute position* information due to *padding effect* [75].

3.1.3 Evaluating Large-kernels ConvNets

The third reason to abandon large kernels, even though the large-kernel ConvNet is designed properly, is that its ImageNet accuracy looks no better than a small-kernel ConvNet. However, Table 2b (after *re-param*) shows increasing the kernel size of MobileNet V2 from 3×3 to 9×9 improves the *ImageNet* accuracy by 1.33%, but the *Cityscapes* mIoU by 3.99%. Such a phenomenon indicates that models of similar ImageNet scores could have very different capabilities in downstream tasks.

Remark. What causes the phenomenon? First, large kernel design significantly increases the *Effective Receptive Fields (ERFs)* [42], as shown in Figure 2. Numerous works have demonstrated "contextual" information, which implies large ERFs, is crucial in many downstream tasks like object detection and semantic segmentation [39], [76], [77], [78], [79]. Second, We deem another reason might be that large kernel design contributes more shape biases to the network. Briefly speaking, ImageNet pictures can be correctly classified according to either texture or shape, as proposed

in [80], [81]. However, humans recognize objects mainly based on shape cue rather than texture, therefore a model with stronger shape bias may transfer better to downstream tasks. A recent study [51] points out ViTs are strong in shape bias, which partially explains why ViTs are super powerful in transfer tasks. In contrast, conventional CNNs trained on ImageNet tend to bias towards texture [80], [81]. Fortunately, we find that simply enlarging the kernel size in ConvNets can effectively improve the shape bias, which means a large-kernel model makes decisions based more on the shapes of objects than the textures.

Therefore, we propose another guideline regarding the evaluation of large-kernel ConvNets.

Guideline 5: evaluate large-kernel ConvNets by the performance of downstream tasks.

3.2 Step 2: Designing a Competitive Large-Kernel Architecture

As discussed above, we have been aware of five basic guidelines for making large kernels practical and then seek to explore how to design a powerful and competitive large-kernel architecture.

We first construct a vanilla architecture as a baseline to verify which design choices work well with large kernels.

Vanilla architecture. As a common practice, the main body of the model is split into four stages connected by downsampling blocks. Specifically, the first downsampling block uses two stride-2 3×3 convolutional layers to transform the raw input into C-channel feature maps, where C is an architectural hyper-parameter. The other three downsampling blocks each use one stride-2 3×3 conv layer performing 2× channel expansion so that the numbers of channels in the four stages are C, $2C$, $4C$, and $8C$, respectively. A stage comprises blocks whose vanilla design resembles ConvNeXt, *i.e.*, a *depthwise (DW)* conv layer and a *Feed-Forward Network (FFN)* with GRN unit [47]. However, we use Batch Normalization (BN) instead of Layer Normalization [82] after the conv layer as BN can be equivalently merged into the conv layer to eliminate its inference costs. We use another BN after the FFN, which can also be equivalently merged into the preceding layer (*i.e.*, the second linear layer in FFN). The numbers of such blocks in the four stages are denoted by $\boldsymbol{N} := (N_1, N_2, N_3, N_4)$. Following ConvNeXt-T, the vanilla architecture uses $C = 96$ and $\boldsymbol{N} = (3, 3, 9, 3)$. By default, the last three stages use DW 13×13 as the convolutional layer, and the first stage uses DW 3×3.

Experimental settings and metrics. According to Guideline 5, large-kernel ConvNets should be evaluated on downstream tasks, as their full potential may not be accurately reflected by ImageNet accuracy alone. Therefore, in addition to reporting the ImageNet-1K accuracy after 100 epochs of training, we transfer the trained model with UPerNet [83] to ADE20K to evaluate its performance on semantic segmentation. We report the single-scale mIoU after a 160k-iteration standard finetuning process [72]. Besides the parameters and FLOPs, we test the actual throughput on an A100 GPU with a batch size of 128 and an input resolution of 224×224, measured in images per second (img/s). See the Appendix for detailed configurations.

We then discuss and verify a series of design choices made in a large-kernel ConvNets. In the following, we summarize our conclusion as a guideline and present the experimental evidence.

3.2.1 Block Design for Large-Kernel ConvNets

Guideline 6: regarding the block design, use efficient structures that perform both inter-channel communications and spatial aggregations to increase the depth. We first aim to enhance the model's representational capacity by universally incorporating structures that provide nonlinearity and efficient trainable transformations. To achieve this, we employ a bottleneck consisting of a 1×1 conv that reduces the channels to 1/4, followed by a DW 3×3 conv, and another 1×1 conv to expand the channels back (Fig. 7). BN and ReLU are applied after each conv layer as standard practice. As shown in Table 3a, this approach improves performance with acceptable overhead (+1.2 mIoU with a 12% slowdown). Performance degrades when we remove the DW 3×3 conv, leaving only two 1×1 conv layers, or when we replace the bottleneck structure with two DW 3×3 layers. This indicates that effective structures require both spatial aggregation transformations and channel mixing. Motivated by this, considering that SE Block [63] elegantly realizes both transformations in a more efficient way (*i.e.*, global average pooling and nonlinear mapping of the pooled vectors), we try it also with 1/4 channel reduction and observe a better performance and higher throughput.

We therefore use the SE Block as a substructure of our block design in the following explorations.

3.2.2 Micro Design with Structural Re-parameterization for Large-Kernel ConvNets

Guideline 7: use dilated small kernels to re-parameterize a large kernel. We then explore the micro (*i.e.*, layer-level) design for large-kernel ConvNet. According to Guideline 3, we should use a parallel small-kernel conv together with a large-kernel layer, as the former helps capture the small-scale patterns during training. Former discussions, however, have primarily focused on simple methods that make large kernels more practical and on explaining the underlying mechanism, rather than offering a competitive solution for building a powerful large-kernel architecture. While we now aim at the latter goal, we recognize that simply using a small kernel to re-parameterize a large kernel may not be optimal, as both capture dense patterns despite their different receptive fields. More than that, we reckon that, except for small-scale patterns, enhancing the large kernel's capability to capture *sparse patterns* (*i.e.*, a pixel on a feature map may be more related to some distant pixels than its neighbors) may yield features of higher quality. The need to capture such patterns exactly matches the mechanism of dilated convolution - from a sliding-window perspective, a dilated conv layer with a dilation rate of r scans the input channel to capture spatial patterns where each pixel of interest is $r - 1$ pixels away from its neighbor. Therefore, we use dilated conv layers parallel to the large kernel and add up their outputs.

To eliminate the inference cost of the extra dilated conv layers, we propose to equivalently transform the whole block into a single non-dilated conv layer for inference.

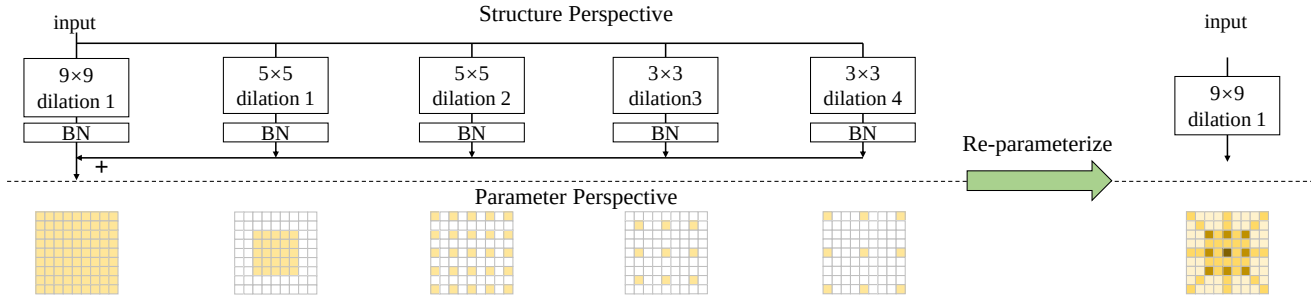

Fig. 6: Dilated Reparam Block (§ 3.2.2) uses dilated small-kernel conv layers to enhance a non-dilated large-kernel layer. Such dilated layers are equivalent to a non-dilated conv layer with a larger sparse kernel, as shown from the parameter perspective so that the whole block can be equivalently transformed into a single large-kernel conv. This example shows K=9, and we may use more dilated layers for larger K.

TABLE 3: **Comparisons among design choices of Step 2**. We report the Top-1 Accuracy (%) and mIoU (%) on the ImageNet-1k and ADE-20K datasets.

(a) Different efficient extra structures to increase the depth.

Extra structure	Params	FLOPs	Img/s	Acc	mIoU
None	31.3M	4.92G	1954	81.2	45.1
(A) Bottleneck	32.9M	5.18G	1716	81.5	46.3
(B) Two 1×1	32.9M	5.17G	1745	81.3	46.2
(C) Two DW 3×3	31.4M	4.96G	1659	81.3	45.4
(D) SE Block	32.9M	4.92G	1863	**81.6**	**46.5**

(b) Different forms of Structural Re-parameterization on the 13×13 conv layers based on the Vanilla architecture.

Re-param	$\boldsymbol{k}$	$\boldsymbol{r}$	Acc	mIoU
None	N/A	N/A	81.44±0.04	45.78±0.05
Dilated Reparam	5,7,3,3,3	1,2,3,4,5	81.63±0.02	**46.37**±0.10
Same kernel size	5,7,3,3,3	1,1,1,1,1	81.55±0.01	46.07±0.07
Same eq kernel size	5,13,7,9,11	1,1,1,1,1	81.59±0.02	46.17±0.04

(c) Different kernel sizes in the four stages denoted by S1 - S4.

S1	S2	S3	S4	Params	FLOPs	Img/s	Acc	mIoU
3	13	13	13	32.9M	4.92G	1863	81.6	**46.5** (42.4)
3	11	11	11	32.6M	4.86G	1876	81.6	45.5 (41.9)
3	3	13	13	32.8M	4.85G	2006	81.7	46.1
3	13	3	13	32.4M	4.81G	2015	81.6	45.9
3	13	13	3	32.5M	4.90G	1884	81.4	45.8
3	15	15	15	33.3M	4.99G	1851	81.7	45.9 (**42.7**)
13	13	13	13	33.0M	5.06G	1547	81.6	44.9 (42.4)

(d) Different numbers of Large-Kernel and Small-Kernel Blocks in Stage 3.

N_3	LarK	SmaK	Params	FLOPs	Img/s	Acc	mIoU
9	9	0	32.9M	4.92G	1863	81.6	**46.5**
27	27	0	56.7M	9.31G	1145	82.3	49.0
27	14	13, 3×3	55.9M	9.15G	1229	82.3	48.8
27	9	18, 3×3	55.6M	9.10G	1264	82.3	48.8
27	9	18, w/o 3×3	55.5M	9.08G	1289	82.2	47.8

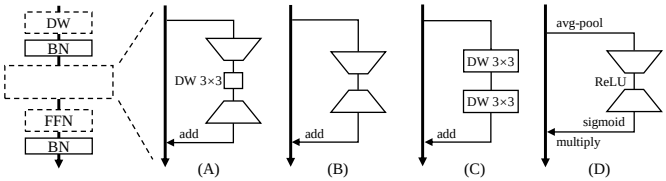

Fig. 7: Options of the extra structures to increase the depth.

Since *ignoring pixels of the input is equivalent to inserting extra zero entries into the conv kernel, a dilated conv layer with a small kernel can be equivalently converted into a non-dilated* (i.e., $r = 1$) *layer with a sparse larger kernel*. Let k be the kernel size of the dilated layer, by inserting zero entries, the kernel size of the corresponding non-dilated layer will be $(k-1)r+1$, which is referred to as the *equivalent kernel size* for brevity. We further note that such transformation from the former kernel $\mathrm{W} \in \mathcal{R}^{k\times k}$ to the latter $\mathrm{W}' \in \mathcal{R}^{((k-1)r+1)\times((k-1)r+1)}$ can be elegantly realized by a transpose convolution with a stride of r and an identity kernel $\mathrm{I} \in \mathcal{R}^{1\times 1}$, which is scalar 1 but viewed as a kernel tensor. [4] With PyTorch-style pseudo code, that is

$$\mathrm{W}' = \texttt{conv_transpose2d}(\mathrm{W}, \mathrm{I}, \text{stride} = r)\,. \qquad (1)$$

The equivalency can be easily verified - given an arbitrary $\mathrm{W} \in \mathcal{R}^{k\times k}$ and an arbitrary input channel, a convolution with W and a dilation rate r always yields identical results to a non-dilated convolution with W′. [5]

Based on such equivalent transformations, we propose a novel module named *Dilated Reparam Block*, which uses a non-dilated small-kernel and multiple dilated small-kernel layers to enhance a non-dilated large-kernel conv layer. Its hyper-parameters include the size of large kernel K, the size of parallel conv layers $\boldsymbol{k}$, and the dilation rate $\boldsymbol{r}$. The shown case (Fig. 6) with four parallel layers is denoted by K=9, $\boldsymbol{r}$=(1, 2, 3, 4), $\boldsymbol{k}$=(5, 5, 3, 3). For a larger K, we may use more dilated layers with larger kernel sizes or dilation rates. The kernel sizes and dilation rates of the parallel branches are flexible, and the only constraint is $(k-1)r+1 \leq K$. For example, with K=13 (the default setting in our experiments), we use five layers with $\boldsymbol{k}$=(5, 7, 3, 3, 3), $\boldsymbol{r}$=(1, 2, 3, 4, 5), so the equivalent kernel sizes will be (5, 13, 7, 9, 11), respectively. To convert a Dialted Reparam Block into a large-kernel conv layer for inference, we first merge every BN into the preceding conv layer, convert every layer with dilation $r > 1$ with function 1, and add up all the resultant kernels with appropriate zero-paddings. For example, the layer in Fig. 6 with k=3, r=3 is converted into a sparse 7×7 kernel and added to the 9×9 kernel with one-pixel zero paddings on each side. For a fair comparison with Dilated

4. We showcase a single-channel conv and it is easy to generalize the transformation to multi-channel cases. See the Appendix for details.

5. In common cases where the output and input have the same size, *i.e.*, the padding of the former is $\frac{k-1}{2}$, note the padding of the latter should be $\frac{(k-1)r}{2}$ since the size of the equivalent sparse kernel is $(k-1)r+1$.

Reparam Block, we try two variants with the same numbers of parallel branches composed of non-dilated layers with **A)** the same kernel sizes or **B)** the same equivalent kernel sizes. For our default setting of K=13, $\boldsymbol{r}$=(1, 2, 3, 4, 5), $\boldsymbol{k}$=(5, 7, 3, 3, 3), the kernel sizes of the five branches will be $\boldsymbol{k}$=(5, 7, 3, 3, 3) or (5, 13, 7, 9, 11) for the two variants, respectively. All the models end up with the same inference structure but the training structures differ. Table 3b shows lower performance of variants, suggesting that large kernel benefits from the parallel dilated conv layers' abilities to capture sparse patterns, rather than merely the extra small kernels (variant A) or the combination of different receptive fields (variant B).

We therefore use Dilated Reparam Block by default. [6]

3.2.3 Kernel Size of Large-Kernel ConvNets

Guideline 8: decide kernel size according to the downstream task and usually use large kernels in middle- and high-level layers. As introduced above, the vanilla architecture uses 3×3 conv in the first stage and 13×13 in the last three stages. Table 3c shows that replacing the large kernels in the last three stages with 3×3 or changing K from 13 to 11 degrades the models, especially in the ADE20K mIoU, which highlights the significance of large kernels. Interestingly, using 13×13 in Stage 1 or enlarging K from 13 to 15 makes almost no difference in the ImageNet accuracy but reduces the ADE20K mIoU.

Remark. We argue that this phenomenon does not mean larger kernels result in lower feature quality. It is due to the structural priors of UPerNet, which takes the features extracted by the low-level layers of the backbone and assumes they should only encode local information so that combining them with the high-level features extracted from the last layers of the backbone results in better segmentation. With larger kernels in lower stages, the low-level features are no longer confined to small local areas, so the UPerNet benefits less from combining them with the high-level features. We verify this explanation by making the UPerNet only use the high-level features (*i.e.*, outputs of Stage 4) to evaluate the quality of the eventual features alone. Under this setting, K=15 delivers the best mIoU (42.7), the model with large kernels in Stage 1 performs as well as the baseline (42.4), and K=11 performs the worst (41.9). Such observations confirm that large kernels, even when they are used inappropriately, *do not damage the feature quality* of ConvNet but *merely make the low-level features less favorable for certain downstream models that require local low-level features*, suggesting we should decide the kernel size according to the specific downstream tasks and framework.

Considering this, we employ 13×13 kernels in the middle- and high-level stages by default.

3.2.4 Scaling Rule of Large-Kernel ConvNets

Guideline 9: while scaling up the depth, the added blocks should use small kernels. The scaling rule of existing large-kernel ConvNets follows the traditional ConvNets, *i.e.*, stacking more large kernels to build up a deeper model, but we argue that a large-kernel ConvNet may not benefit from more large kernels. In this group of experiments (Table 3d), we scale up N_3 from 9 to 27, following ConvNeXt-S [16]. Considering that nine 13×13 blocks may have already built up sufficient receptive field, we examine if the added blocks should also use large kernels. Specifically, we refer to the block with a Dilated Reparam Block as the *Large Kernel Block (LarK Block)* and name a block that uses a DW 3×3 conv as a *Small Kernel Block (SmaK Block)* so that there are 3 SmaK Blocks in Stage 1 and 3/9/3 LarK Blocks in Stage 2/3/4 of the shallow model. While scaling up the depth of Stage 3, we tried the following options. **A)** All of the 27 blocks are LarK Blocks. **B)** We interleave SmaK with LarK Blocks so that Stage 3 has 14 LarK Blocks and 13 SmaK Blocks. **C)** We place two SmaK Blocks after a LarK Block so that the resultant model will have the same 9 LarK Blocks as before but 18 extra SmaK Blocks. **D)** We remove the DW 3×3 layers in SmaK Blocks. Table 3d shows that scaling up the depth brings significant improvements, which is expected, and 9 LarK Blocks are sufficient. Though 27 LarK Blocks perform slightly better in the ADE20K mIoU, the inference speed is observably slowed down. Besides, the model without 3×3 conv in SmaK Blocks shows significantly lower mIoU with only minor improvements in the throughput, suggesting such small kernels in SmaK Blocks are useful while scaling up the depth of large-kernel ConvNet as they increase the abstract hierarchy of spatial patterns, though they may not effectively enlarge the ERF [1], [42]. This observation supports our motivation to decouple the effects of conv layers in enlarging the ERF and extracting more complicated spatial patterns, as discussed in Sec. 1.

3.2.5 Architectural Specifications

Following our proposed guidelines, we instantiate a series of models (Table 4). For a fair comparison with ConvNeXt V2 [47], UniRepLKNet-A/F/P/N follows its configurations. We scale up the depth to build UniRepLKNet-T/S and scale up the width to construct UniRepLKNet-S/B/L/XL/H.

TABLE 4: Architectural hyper-parameters of UniRepLKNet instances, including the number of blocks in the four stages N_1, N_2, N_3, N_4 and channels C of the first stage. Stage 1 uses SmaK Blocks, and Stages 2 and 4 use LarK Blocks only. For Stage 3, *e.g.*, "9 + 18" means 9 LarK Blocks and 18 SmaK Blocks.

	N_1	N_2	N_3	N_4	C	Params
UniRepLKNet-A	2	2	6 + 0	2	40	4.4M
UniRepLKNet-F	2	2	6 + 0	2	48	6.2M
UniRepLKNet-P	2	2	6 + 0	2	64	10.7M
UniRepLKNet-N	2	2	8 + 0	2	80	18.3M
UniRepLKNet-T	3	3	9 + 9	3	80	31.0M
UniRepLKNet-S	3	3	9 + 18	3	96	55.6M
UniRepLKNet-B	3	3	9 + 18	3	128	97.9M
UniRepLKNet-L	3	3	9 + 18	3	192	218.3M
UniRepLKNet-XL	3	3	9 + 18	3	256	386.4M
UniRepLKNet-H	3	3	9 + 18	3	480	1.4B

3.3 Step 3: Generalizing Large-Kernel ConvNets to Multiple Modalities

To utilize the universal perception ability of UniRepLKNet, we preprocess the data of different modalities into $B \times$

6. While this paper describes the architecture, using a $K \times K$ ($K \geq 9$) conv means a $K \times K$ Dilated Reparam Block, unless otherwise noted.

$C' \times H \times W$ embedding maps, where B is the batch size and C' is determined by the modality, and configure the input channel of the first layer of UniRepLKNet to C'. For simplicity, the other parts of the models are the same as the UniRepLKNet initially designed for the image without any modality-specific customization.

Time-series. Let L and D be the length and dimensions of a time-series sequence $\boldsymbol{x}_T \in \mathbb{R}^{B\times L\times D}$, we adopt the embedding layer in Corrformer [46] to split it into n nodes then project it into a latent space $\mathbb{R}^{Bn\times L\times D'}$ (D' and n are configurable hyper-parameters of the embedding layer). Then we simply reshape it into a single-channel embedding map:

$$\begin{aligned}\boldsymbol{x}_T \in \mathbb{R}^{B\times L\times D} &\to \mathbb{R}^{Bn\times L\times \frac{D}{n}} \to \mathbb{R}^{Bn\times L\times D'} \\ &\to \mathbb{R}^{Bn\times 1\times H\times W} \text{ s.t. } HW = LD'.\end{aligned} \tag{2}$$

Audio. Let T and F be the numbers of time frames and frequency bins, we use $\boldsymbol{x}_A \in \mathbb{R}^{B\times T\times F}$ to represent audio data. A sample is seen as a $1 \times T \times F$ embedding map that resembles a single-channel image so C'=1, H=T, W=F.

$$\boldsymbol{x}_A \in \mathbb{R}^{B\times T\times F} \to \mathbb{R}^{B\times 1\times T\times F}. \tag{3}$$

Point cloud. Assume a sample comprises P points each represented by the X/Y/Z coordinates, we use a series of conv layers to generate three-view projections [28]. We configure the resolution of the generated projections to be 224 so that H=W=224, C'=3.

$$\boldsymbol{x}_P \in \mathbb{R}^{B\times P\times 3} \to \mathbb{R}^{B\times 3\times 224\times 224}. \tag{4}$$

Video. We represent a video as N_F frames and each frame is a $3 \times h \times w$ image. We reshape it by merging the frame dimension into the height and width dimensions so that we obtain a representation that can be viewed as a single image created by laying out (*i.e.*, concatenating) the N_F frames. For example, in our experiments, we have N_F=16 and h=w=224 so that H=W=896. Generally,

$$\boldsymbol{x}_V \in \mathbb{R}^{B\times N_F\times 3\times h\times w} \to \mathbb{R}^{B\times 3\times H\times W} \text{s.t.} \frac{HW}{hw} = N_F. \tag{5}$$

3.4 Step 4: Fusing Multimodal Features with Large Kernel Convolution

In addition to extracting features, we further explore large-kernel convolution to fuse multimodal features as cross-attention [84]. Inspired by the flexibility of asymmetric convolution in fusing features of diverse shapes [85], we propose the asymmetric large-kernel convolution to broadly fuse features across diverse shapes and modalities. As cross-attention mechanism to fuse two features of $\boldsymbol{X}$ and $\boldsymbol{Y}$, where $\boldsymbol{X} \in \mathbb{R}^{L_1\times D}, \boldsymbol{Y} \in \mathbb{R}^{L_2\times D}$, L_1 and L_2 denote the length of a sequence of tokens, D denotes the feature dimension (Note that the feature map $\boldsymbol{X} \in \mathbb{R}^{L_1\times D}$ can be easily reshaped as $\boldsymbol{X} \in \mathbb{R}^{H\times W\times C}$).

The asymmetric large-kernel convolution uses one feature map as the convolutional kernel to convolve another feature map, allowing for dynamic and context-aware fusion of multimodal features. Specifically, the convolution operation is performed by treating $\boldsymbol{Y}$ as the convolutional kernel that is applied to $\boldsymbol{X}$. In this setup, each element of $\boldsymbol{Y}$ serves as a dynamic filter that modulates $\boldsymbol{X}$ according to its contextual information. The output feature map $\boldsymbol{Z}$ can be expressed as:

$$\boldsymbol{Z}_{i,j} = \sum_{k=1}^{L_2} \boldsymbol{X}_{i+k-1} \cdot \boldsymbol{Y}_{k,j},$$

where $\boldsymbol{Z}_{i,j}$ represents the correlation between $\boldsymbol{X}$ starting at position i with the filter defined by $\boldsymbol{Y}_j$. This approach allows $\boldsymbol{X}$ to be dynamically influenced by the patterns in $\boldsymbol{Y}$, facilitating an adaptive and effective fusion of the two feature maps. It efficiently captures the intrinsic correlation between the features, making it a computationally efficient alternative for multimodal feature fusion tasks.

4 Experiments

4.1 Experiments for Visual Recognition

ImageNet classification. Following ConvNeXt [16], we use the widely adopted 300-epoch receipt to train UniRepLKNet-A/F/P/N/T/S on ImageNet-1K; we pre-train UniRepLKNet-S/B/L/XL on ImageNet-22K using the 90-epoch receipt and fine-tune with ImageNet-1K for 30 epochs (see the Appendix for details). As our goal is to develop models that *run with high actual speed*, we evaluate the actual throughput on the same A100 GPU using a batch size of 128. Table 5 shows the top-1 accuracy on the ImageNet-1K validation set where the results are sorted by the throughput. We split the results into seven segments for better readability. **1)** UniRepLKNet-A/F outperforms ConvNeXt-V2-A/F by 0.8/0.6 in the accuracy and runs 19%/17% faster, respectively. **2)** UniRepLKNet-P/N outperforms FastViT-T12/S12 and ConvNeXt V2-P/N by clear margins. **3)** UniRepLKNet-T outperforms multiple small-level competitors. **4)** UniRepLKNet-S outperforms a series of small-level and even base-level models in both speed and accuracy and runs almost as fast as InternImage-T. **5)** With ImageNet-22K pretraining, UniRepLKNet-S even approaches the accuracy of RepLKNet-31L and runs 3× as fast as the latter. UniRepLKNet-B outperforms CoAtNet-2 and DeiT III-B by clear margins. UniRepLKNet-L outperforms InternImage-L in both accuracy and throughput. **6)** On the XL-level, UniRepLKNet-XL outperforms in both accuracy and throughput, running more than 2× as fast as CoAtNet-3 and 3× as DeiT III-L.

COCO object detection and instance segmentation. We transfer the pretrained UniRepLKNets as the backbones of Cascade Mask R-CNN [102], [103] and adopt the standard 3x (36-epoch) training configuration with MMDetection [104]. Table 6 shows UniRepLKNet outperforms Swin, ConvNeXt, RepLKNet, and SLaK, which are representatives of ViTs, modern medium-kernel ConvNets, and existing large-kernel ConvNets, respectively, and shows comparable performance to InternImage [88], which is a latest powerful architecture with deformable convolution.

ADE20K semantic segmentation. We use the pretrained UniRepLKNets as the backbones of UPerNet [83] on ADE20K [105] and adopt the standard 160k-iteration training receipt with MMSegmentation [72]. Table 7 reports the mIoU on the validation set. Impressively, UniRepLKNet outperforms InternImage and the other models.

TABLE 5: **ImageNet classification**. Throughput is tested with an A100 GPU and batch size of 128. "T/C" denote transformer/ConvNet. "‡" indicates ImageNet-22K [43] pretraining.

Method	Type	Input size	Params (M)	FLOPs (G)	Throughput (img/s)	Acc (%)
UniRepLKNet-A	C	224^2	4.4	0.6	**5942**	**77.0**
UniRepLKNet-F	C	224^2	6.2	0.9	**5173**	**78.6**
ConvNeXt V2-A [47]	C	224^2	3.7	0.5	5054	76.2
FastViT-T8 [48]	T	256^2	3.6	0.7	5025	75.6
ConvNeXt V2-F [47]	C	224^2	5.2	0.8	4329	78.0
UniRepLKNet-P	C	224^2	10.7	1.6	**3949**	**80.2**
FastViT-T12 [48]	T	256^2	6.8	1.4	3407	79.1
ConvNeXt V2-P [47]	C	224^2	9.1	1.4	3339	79.7
FastViT-S12 [48]	T	256^2	8.8	1.8	3162	79.8
UniRepLKNet-N	C	224^2	18.3	2.8	**2807**	**81.6**
ConvNeXt V2-N [47]	C	224^2	15.6	2.4	2405	81.2
UniRepLKNet-T	C	224^2	31	4.9	**1804**	**83.2**
FastViT-SA24 [48]	T	256^2	21	3.8	1670	82.6
PVTv2-B2 [86]	T	224^2	25	4.0	1620	82.0
CoAtNet-0 [87]	T	224^2	25	4.2	1613	81.6
DeiT III-S [50]	T	224^2	22	4.6	1485	81.4
SwinV2-T/8 [49]	T	256^2	28	6	1406	81.8
SLaK-T [40]	C	224^2	30	5.0	1312	82.5
InternImage-T [88]	C	224^2	30	5	1292	83.5
UniRepLKNet-S	C	224^2	56	9.1	**1265**	**83.9**
ConvNeXt-S [16]	C	224^2	50	8.7	1182	83.1
HorNet-T [89]	C	224^2	23	3.9	1162	83.0
FastViT-SA36 [48]	T	256^2	30	5.6	1151	83.6
CoAtNet-1 [87]	T	224^2	42	8.4	969	83.3
SLaK-S [40]	C	224^2	55	9.8	967	83.8
FastViT-MA36 [48]	T	256^2	43	7.9	914	83.9
SwinV2-S/8 [49]	T	256^2	50	12	871	83.7
RepLKNet-31B [1]	C	224^2	79	15.3	859	83.5
PVTv2-B5 [86]	T	224^2	82	11.8	802	83.8
UniRepLKNet-S‡	C	384^2	56	26.7	**435**	**86.4**
ConvNeXt-S‡ [16]	C	384^2	50	25.5	415	85.8
UniRepLKNet-B‡	C	384^2	98	47.2	**314**	**87.4**
ConvNeXt-B‡ [16]	C	384^2	89	45.1	304	86.8
UniRepLKNet-L‡	C	384^2	218	105.4	**190**	**87.9**
ConvNeXt-L‡ [16]	C	384^2	198	101	185	87.5
CoAtNet-2‡ [87]	T	384^2	75	49.8	163	87.1
RepLKNet-31L‡ [1]	C	384^2	172	96.0	158	86.6
InternImage-L‡ [88]	C	384^2	223	108	143	87.7
DeiT III-B‡ [50]	T	384^2	87	55.5	138	86.7
UniRepLKNet-XL‡	C	384^2	386	187	**131**	**88.0**
ConvNeXt-XL‡ [16]	C	384^2	350	179	129	87.8
HorNet-L‡ [89]	C	384^2	202	102	127	87.7
InternImage-XL‡ [88]	C	384^2	335	163	114	88.0
CoAtNet-3‡ [87]	T	384^2	168	107	103	87.6
SwinV2-L/24‡ [49]	T	384^2	197	115	88	87.6
CoAtNet-4‡ [87]	T	384^2	275	190	58	87.9
DeiT III-L‡ [50]	T	384^2	305	191	42	87.7

4.2 Universal Perception on More Modalities

Time-series. Following Corrformer [46], we conduct experiments on the Global Temperature and Wind Speed Forecasting challenge [7] using the dataset collected from the National Centers for Environmental Information (NCEI), GFS [8] stands for the Global Forecasting System. This huge-scale dataset contains hourly averaged wind speed and temperature data from 3,850 stations with different geographical scales and densities, spanning from 2019 to 2021. For a fair comparison with Corrformer [46], which was the previous state-of-the-art method, we use its embedding layer (as introduced in Sec. 3.3) and decoder and only replace its encoder transformer with UniRepLKNet-S. We also compare UniRepLKNet-S against a wide range of methods, including statistical and numerical approaches. Table 12 shows UniRepLKNet delivers a new state-of-the-art forecasting precision, achieving the lowest errors of 7.602, 1.832, 3.865, and 1.301 for MSE and MAE in forecasting global temperature and wind speed, respectively, with fewer parameters than existing deep learning methods. It is particularly noteworthy that UniRepLKNet, a generalist model, outperforms time-series specialists such as Pyraformer [113] and Corrformer [46] in both precision and efficiency. The significant advantages of UniRepLKNet are that it opens up new avenues for architectural discussions in time-series forecasting and presents a viable alternative to transformer

7. https://codeocean.com/capsule/0341365/tree/v1
8. https://www.ncei.noaa.gov/

TABLE 6: **Object detection on COCO validation set**. FLOPs are measured with 1280×800 inputs. "‡" ImageNet-22K pretraining.

Method	Params (M)	FLOPs (G)	AP^{box}	AP^{mask}
UniRepLKNet-T	89	749	**51.8**	**44.9**
Swin-T [41]	86	745	50.4	43.7
ConvNeXt-T [16]	86	741	50.4	43.7
SLaK-T [40]	-	-	51.3	44.3
UniRepLKNet-S	113	835	**53.0**	**45.9**
Swin-S [41]	107	838	51.9	45.0
ConvNeXt-S [16]	108	827	51.9	45.0
UniRepLKNet-S‡	113	835	**54.3**	**47.1**
UniRepLKNet-B‡	155	978	**54.8**	**47.4**
Swin-B‡ [41]	145	982	53.0	45.8
ConvNeXt-B‡ [16]	146	964	54.0	46.9
RepLKNet-31B‡ [1]	137	965	52.2	45.2
UniRepLKNet-L‡	276	1385	55.8	48.4
Swin-L‡ [41]	253	1382	53.9	46.7
ConvNeXt-L‡ [16]	255	1354	54.8	47.6
RepLKNet-31L‡ [1]	229	1321	53.9	46.5
InternImage-L‡ [88]	277	1399	**56.1**	**48.5**
UniRepLKNet-XL‡	443	1952	**56.4**	**49.0**
InternImage-XL‡ [88]	387	1782	56.2	48.8
ConvNeXt-XL‡ [16]	407	1898	55.2	47.7

TABLE 7: **Semantic segmentation on ADE20K validation set**. The FLOPs are measured with 512×2048 or 640×2560 inputs according to the crop size. "SS" and "MS" mean single- and multi-scale testing, respectively. "‡" ImageNet-22K [43] pretraining.

Method	Crop size	Params (M)	FLOPs (G)	mIoU (SS)	mIoU (MS)
UniRepLKNet-T	512^2	61	946	**48.6**	**49.1**
Swin-T [41]	512^2	60	945	44.5	45.8
ConvNeXt-T [16]	512^2	60	939	46.0	46.7
SLaK-T [40]	512^2	65	936	47.6	-
InternImage-T [88]	512^2	59	944	47.9	48.1
UniRepLKNet-S	512^2	86	1036	**50.5**	**51.0**
Swin-S [41]	512^2	81	1038	47.6	49.5
ConvNeXt-S [16]	512^2	82	1027	48.7	49.6
SLaK-S [40]	512^2	91	1028	49.4	-
InternImage-S [88]	512^2	80	1017	50.1	50.9
UniRepLKNet-S‡	640^2	86	1618	**51.9**	**52.7**
UniRepLKNet-B‡	640^2	130	1850	**53.5**	**53.9**
Swin-B‡ [41]	640^2	121	1841	50.0	51.7
ConvNeXt-B‡ [16]	640^2	122	1828	52.6	53.1
RepLKNet-31B‡ [1]	640^2	112	1829	51.5	52.3
UniRepLKNet-L‡	640^2	254	2507	**54.5**	**55.1**
Swin-L‡ [41]	640^2	234	2468	52.1	53.5
RepLKNet-31L‡ [1]	640^2	207	2404	52.4	52.7
ConvNeXt-L‡ [16]	640^2	235	2458	53.2	53.7
InternImage-L‡ [88]	640^2	256	2526	53.9	54.1
UniRepLKNet-XL‡	640^2	425	3420	**55.2**	**55.6**
ConvNeXt-XL‡ [16]	640^2	391	3335	53.6	54.0
InternImage-XL‡ [88]	640^2	368	3142	55.0	55.3

TABLE 8: **Audio recognition** on Speech Commands V2 (SPC-2) and AudioSet-2M (AS-2M) datasets.

Method	Pretrain	Type	SPC-2 (%)	AS-2M (%)	Params
PANNS [90]	-	ConvNet	61.8	43.1	-
PSLA [91]	IN-1K	ConvNet	96.3	44.4	-
AST [30]	AS-2M	Transformer	96.2	45.9	86.9M
SSAST [92]	AS-2M	Transformer	97.8	-	89.3M
Audio-MAE [93]	AS-2M	Transformer	98.3	47.3	86.2M
Meta-Transformer [28]	LAION-2B	Transformer	97.0	-	86.6M
UniRepLKNet-S	-	ConvNet	**98.5**	**48.5**	**55.5M**

TABLE 9: **Video recognition** accuracy on Kinetics-400 tasks.

Method	Pretrain	Type	Acc (%)	Params
Specialist				
SlowFast-101 [94]	IN-1K	ConvNet+RNN	79.8	62.8M
MViTv2-B [32]	IN-1K	Transformer	81.2	51.2M
TimeSFormer [95]	K400	Transformer	80.7	122M
Generalist				
Meta-Transformer [28]	LAINON-2B	Transformer	47.3	86.9M
ImageBind [27]	CLIP Data	Transformer	50.0	632M
UniRepLKNet-S	-	ConvNet	54.8	55.5M

TABLE 10: **Point cloud analysis** on ModelNet-40 and Objaverse-LVIS datasets.

Method	Type	ModelNet-40		Objaverse-LVIS		
		mAcc (%)	OA (%)	Top1	Top3	Top5
PointNet [96]	MLP	86.0	89.2	-	-	-
PointNet++ [97]	MLP	-	91.9	-	-	-
PointConv [98]	ConvNet	-	92.5	-	-	-
KPConv [99]	ConvNet	-	92.9	-	-	-
DGCNN [100]	ConvNet	90.2	92.9	-	-	-
OpenShape [101]	Transformer	83.4	-	43.4	64.8	72.4
UniRepLKNet-S	ConvNet	**90.3**	**93.2**	**50.3**	**71.6**	**78.2**

TABLE 11: Universal perception performance with other ConvNets or UniRepLKNet with a smaller kernel size.

Modality	Time-Series	Point Cloud	Audio	Video
	MAE↓	OA (%)	Acc (%)	Acc (%)
ResNet-101 [7] (K=3)	7.846	92.6	73.6	41.3
ConvNeXt-S [16] (K=7)	7.641	92.7	94.3	48.5
UniRepLKNet-S (K=11)	7.751	92.9	94.7	51.7
UniRepLKNet-S (K=13)	**7.602**	**93.2**	**98.5**	**54.8**

models.

Audio. We use Speech Commands V2 [159], which contains 105,829 one-second recordings of 35 common speech commands. Table 8 shows UniRepLKNet seamlessly adapts to audio and delivers an impressive accuracy of 98.5% and 48.5% on AS-2M even without pretraining. Compared to transformers such as AST [30] and Audio-MAE [93], UniRepLKNet stands out with fewer parameters. Compared to previous ConvNets designed for audio, UniRepLKNet achieves better performance without customizations to the structure, highlighting the untapped potential of ConvNets in the realm of audio.

Video. Kinetics-400 [160] contains 240k training videos and 20k validation videos, spanning 400 classes for action recognition. Though the top-1 accuracy of 54.8% is somewhat behind state-of-the-art architectures like MViT [32], we note that UniRepLKNet is a generalist model without pretraining. Compared to the latest generalist methods, ImageBind [27] and Meta-Transformer [28], UniRepLKNet shows higher accuracy and requires no pretraining.

Point cloud. We explore the versatility of UniRepLKNet by assessing its proficiency in learning 3D patterns, extending beyond the conventional 2D signals of images and audio. We use the ModelNet-40 [161] 3D shape classification task with 9,843/2,468 training/validation samples of CAD models from 40 classes. Table 10 shows UniRepLKNet achieves an Overall Accuracy (OA) of 93.2% and a mean Accuracy (mAcc) of 90.3% and 50.3 Top-1 accuracy on the Objaverse-LVIS.

Impact of kernel size on the performance. To investigate the influence of different kernel sizes on performance, we compare UniRepLKNet with models of smaller kernels. We adopted the same modality-specific preprocessing approaches and training configurations for a fair comparison. We take ResNet-101 as a representative small-kernel ConvNet because it has comparable parameters to UniRepLKNet-S. Table 11 shows large kernels are crucial for universal perception, at least in our specific cases.

4.3 Scalable Multimodal Pretraining and Generation

Stage 0: CLIP Pretraining. We utilize the UniRepLKNet-L as the image tower with a standard projection, and follow previous pratice [54], [138] to use a text tower with the same size as ViT-g-14 model pretrained with 11B text samples [138]. The size of the combined image + text CLIP model is 1.4B parameters. UniRepLKNet excels in zero-shot image recognition abilities compared with the same scale models including OpenAI CLIP-L [136], OpenCLIP-L [54], FLIP-L [137], and OpenCLIP-ConvNeXt-L [16], [54] in Table 13 among 26 zero-shot tasks. It's worth noting that our CLIP model shows competitive performance (72.1 *v.s.* 72.4) compared with the EVA-01-CLIP-g/14 model, which has more than 3× parameters than ours.

Stage 1: Large Vision-Language Model (VLM) Pretraining. After CLIP pretraining, we then use pretrained CLIP-UniRepLKNet-L for training large VLMs. Specifically, we use LLaVA-v1.5 [156] as a baseline, which incorporates the text-image alignment and visual instruction process with a convolutional backbone. Specifically, we use LLaVA pretraining data to align Vicuna-7B and UniRepLKNet, then LLaVA-SFT-665k for visual instruction tuning.

As shown in Table 14, UniRepLKNet-Chat-7B demonstrates significant advantages across various benchmarks in Visual Question Answering (VQA), Image Captioning, and multimodal Benchmark tasks. Notably, in the GQA task, UniRepLKNet-Chat-7B scores 59.8, positioning itself competitively among Vision Specialist LLMs. It excels in the VQAv2 task with a remarkable score of 80.2, surpassing models such as Flamingo, InstructBLIP, and IDEALF. Additionally, in the OKVQA task, UniRepLKNet-Chat-7B

TABLE 12: **Time-series forecasting** performance on Global Temperature and Wind Speed Forecasting challenge. UniRepLKNet delivers a new state-of-the-art performance in Mean Squared Error (MSE) and Mean Absolute Error (MAE).

Method	Type	Params	Temperature		Wind speed	
			MSE ↓	MAE ↓	MSE ↓	MAE ↓
Statistics-based						
Holt–Winters [106]	-	-	13.241	2.262	5.912	1.664
Prophet [107]	-	-	11.626	2.946	9.691	2.382
GDBT [NeurIPS'17] [108]	-	-	9.706	2.214	4.101	1.417
Numerical Simulation						
GFS (reanalysis)	-	-	14.933	2.287	9.993	2.340
ERA5 (reanalysis) [109]	-	-	13.448	1.908	4.999	1.587
DeepAR [110]	-	-	32.249	4.262	5.248	1.602
N-BEATS [111]	-	-	9.203	2.117	4.124	1.390
Deep Learning Specialist						
StemGNN [NeurIPS'20] [112]	GNN	180M	13.926	2.746	4.066	1.389
Pyraformer [ICLR'21] [113]	Transformer	158M	23.326	3.669	4.614	1.514
Corrformer [Nat. Mach. Intell.'23] [46]	Transformer	155M	7.709	1.888	3.889	1.304
Generalist						
UniRepLKNet-S	ConvNet	132M	**7.602**	**1.832**	**3.865**	**1.301**

TABLE 13: **Zero-Shot Image Classification performance on 26 datasets with OpenCLIP Pretraining.** We report top-1 accuracy on all datasets. The best results are in **bold** and the second best are underlined.

M ethod	ImageNet-1K [43]	ImageNet-V2 [114]	ImageNet-Adv. [115]	ImageNet-Ren. [116]	ImageNet-Ske. [117]	ObjectNet [118]	CIFAR-10 [119]	CIFAR-100 [119]	MNIST [120]	Caltech101 [121]	SUN397 [122]	FGVC Aircraft [123]	Country-211 [56]	Stanford Cars [124]	DTD [125]	EuroSAT [126]	FER2013 [127]	Flowers-102 [128]	Food-101 [129]	GTSRB [130]	PCam [131]	Pets [132]	Rendered SST2 [56]	RESISC45 [133]	STL-10 [134]	VOC2007 [135]	**avg. top-1 acc.**
OpenAI CLIP-L/14 [136]	75.5	69.9	70.8	87.8	59.6	69.0	95.6	75.8	76.4	86.7	67.6	31.4	**31.9**	77.9	55.4	62.4	49.9	**79.2**	93.1	50.6	52.0	93.5	**68.9**	64.6	**99.4**	67.6	69.7
OpenCLIP-L/14 [54]	76.2	67.8	53.9	87.4	63.3	65.5	96.6	83.3	54.0	85.0	**74.3**	36.3	26.2	92.6	62.9	64.7	53.9	75.8	91.0	56.1	**56.3**	93.1	59.1	66.8	98.8	81.9	70.1
FLIP-L/14 [137]	74.3	66.8	51.2	86.5	59.9	59.1	97.2	84.1	80.3	**93.8**	73.1	29.1	23.1	90.7	60.4	53.5	54.0	75.0	89.3	41.4	50.3	92.6	58.5	**70.8**	98.5	83.1	69.1
EVA-01-CLIP-g/14 [138]	**78.5**	**71.5**	**73.6**	**92.5**	**67.6**	**72.3**	**98.3**	**88.7**	62.6	87.7	74.2	32.4	28.9	91.7	61.7	**73.8**	52.2	74.5	**93.5**	49.3	49.9	**94.2**	58.4	70.3	98.9	**85.7**	**72.4**
OpenCLIP-ConvNeXt-L [54]	75.2	68.2	53.5	87.6	64.3	65.9	96.5	83.1	74.4	84.3	73.0	36.1	25.2	93.2	67.3	69.6	52.9	76.8	90.6	52.8	53.0	92.9	56.2	67.8	98.3	81.3	70.7
OpenCLIP-UniRepLKNet-L [Ours]	76.6	69.5	60.4	88.6	65.0	69.0	96.6	83.1	**80.6**	84.7	73.8	**36.4**	26.5	**93.7**	**68.3**	71.6	53.1	77.3	91.6	**58.2**	48.0	93.8	56.1	**70.8**	98.6	82.5	72.1

TABLE 14: **Evaluation on LLM Benchmarks.** The MLLM evaluation involves 6 VQA tasks (GQA [139], VQAv2 [140], OKVQA [141], TextVQA (TVQA) [142], ScienceQA (SQA) [143] and Vizwiz [144]), 2 image captioning tasks (Nocaps [145] and Flickr30K [146]), and 4 multimodal benchmarks (MME [147], MM Bench (MMB) [148], MMVet [149] and SEED [150]). The LLMs are Chinchilla, Vicuna, Qwen, LLaMA and LLaMA2. The evaluation metrics for VQA and captioning tasks are accuracy and CIDEr, respectively. The results in **bold** and underline are the best and second-best results, respectively.

Method	LLM	Visual Question Answering					Image Caption			MM Benchmark			
		GQA	VQAv2	OKVQA	TVQA	SQA	Vizwiz	NoCaps	Flickr	MME	MMB	MMVet	SEED
Vision Specialist LLM													
Flamingo-9B [151]	Chinchilla-7B	-	51.8	44.7	30.1	-	28.8	-	61.5	-	-	-	-
Flamingo-80B [151]	Chinchilla-70B	-	56.3	50.6	31.8	-	31.6	-	67.2	-	-	-	-
BLIP-2 [152]	Vicuna-7B	-	-	-	40.1	53.8	-	107.5	74.9	-	-	-	-
BLIP-2 [152]	Vicuna-13B	41.0	41.0	-	42.5	61	19.6	103.9	71.6	1293.8	-	22.4	-
InstructBLIP [153]	Vicuna-7B	49.2	-	-	50.1	60.5	34.5	**123.1**	82.4	-	36	26.2	-
InstructBLIP [153]	Vicuna-13B	49.5	-	-	50.7	63.1	34.3	121.9	**82.8**	1212.8	-	25.6	-
IDEFICS-9B [154]	LLaMA-7B	38.4	50.9	38.4	25.9	-	35.5	-	27.3	-	48.2	-	-
IDEFICS-80B [154]	LLaMA-65B	45.2	60.0	45.2	30.9	-	36.0	-	53.7	-	54.5	-	-
Qwen-VL [155]	Qwen-7B	57.5	78.2	56.6	61.5	68.2	38.9	120.2	81.0	1487.5	60.6	-	58.2
LLaVA-v1.5 [156]	Vicuna-7B	**62.0**	78.5	-	58.2	66.8	50.0	-	-	1510.7	64.3	30.5	58.6
Multimodal Generalist LLM													
ImageBind-LLM [157]	LLaMA-7B	41.1	-	-	24.0	51.4	-	29.6	23.5	775.7	-	-	-
AnyMAL-13B [158]	LLaMA2-13B	-	59.6	33.1	24.7	52.7	24.4	-	-	-	-	-	-
AnyMAL-70B [158]	LLaMA2-70B	-	64.2	42.6	32.9	70.8	33.8	-	-	-	-	-	-
OneLLM-7B [29] [CVPR'24]	LLaMA2-7B	59.5	71.6	58.9	34.0	63.4	45.9	115.9	78.6	1392.0	60.0	29.1	61.2
UniRepLKNet-Chat-7B [Ours]	**Vicuna-7B**	59.8	**80.2**	**59.3**	**62.7**	**72.5**	**51.0**	113.5	75.3	**1569.5**	**68.8**	**32.3**	**69.5**

TABLE 15: **Fusing multimodal features with UniRepLKNet**. We evaluate our model on Refer-Davis$_{17}$ [162] dataset following full-video expression and first frame two settings, in terms of region similarity ($\mathcal{J}$), contour accuracy ($\mathcal{F}$) and their average scores $\mathcal{J}\&\mathcal{F}$.

Expression Type	Method	$\mathcal{J}$	$\mathcal{F}$	$\mathcal{J}\&\mathcal{F}$
Full Video	Khoreava *et.al.* [163]	-	-	37.30
	RefVOS(baseline) [162]	-	-	45.10
	CMSA + RNN [139]	36.94	37.23	34.71
	URVOS w/o ft [164]	44.29	49.41	46.85
	URVOS [164]	47.29	55.96	51.45
	ACM [85]	48.39	56.17	52.28
	RefVOS-UniRepLKNet	**50.46**	**57.94**	**54.20**
First Frame	Khoreava *et.al.* [163]	37.30	41.30	39.30
	URVOS [164]	41.23	47.01	44.12
	RefVOS(baseline) [162]	-	-	44.50
	ACM [85]	48.52	56.06	52.29
	RefVOS-UniRepLKNet	**50.63**	**57.72**	**54.17**

achieves a score of 59.3, reflecting its robust performance. The model further distinguishes itself in the TVQA and SQA tasks with accuracy scores of 62.7 and 72.5, respectively, showcasing its strong text understanding and question answering capabilities. Moreover, its outstanding performance is evident in the MME benchmark with a score of 1569.5, highlighting its efficiency in multimodal understanding and reasoning. The model's balanced performance across diverse tasks underscores its versatility and robustness, making it an improved VLM in the field of multimodal large language models.

5 Conclusion

In this paper, the UniRepLKNet shows a leading performance in image recognition and achieves remarkable results on audio and time-series data, outperforming multiple specialist models on those modalities. Traditionally, ConvNets excelled primarily in visual tasks, yet the emergence of Transformer-based architectures had shifted focus away from ConvNets as researchers sought new paradigms for tackling multimodal data. Such results signify a *"**comeback**"* for ConvNet in its original domain and showcase large-kernel ConvNet's potential to *"**conquer**"* new territories. We hope this advancement will inspire further research into large-kernel ConvNets, encouraging new applications and optimizations that extend ConvNets' utility across a broader range of data modalities.

REFERENCES

[1] X. Ding, X. Zhang, J. Han, and G. Ding, "Scaling up your kernels to 31x31: Revisiting large kernel design in cnns," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2022, pp. 11 963–11 975.

[2] X. Ding, Y. Zhang, Y. Ge, S. Zhao, L. Song, X. Yue, and Y. Shan, "Unireplknet: A universal perception large-kernel convnet for audio video point cloud time-series and image recognition," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024, pp. 5513–5524.

[3] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "Imagenet classification with deep convolutional neural networks," in *Advances in neural information processing systems*, 2012, pp. 1097–1105.

[4] C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. Reed, D. Anguelov, D. Erhan, V. Vanhoucke, and A. Rabinovich, "Going deeper with convolutions," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2015, pp. 1–9.

[5] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna, "Rethinking the inception architecture for computer vision," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 2818–2826.

[6] C. Szegedy, S. Ioffe, V. Vanhoucke, and A. A. Alemi, "Inception-v4, inception-resnet and the impact of residual connections on learning," in *Thirty-first AAAI conference on artificial intelligence*, 2017.

[7] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 770–778.

[8] G. Huang, Z. Liu, L. van der Maaten, and K. Q. Weinberger, "Densely connected convolutional networks," in *2017 IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2017, Honolulu, HI, USA, July 21-26, 2017*. IEEE Computer Society, 2017, pp. 2261–2269. [Online]. Available: https://doi.org/10.1109/CVPR.2017.243

[9] Y. LeCun, Y. Bengio *et al.*, "Convolutional networks for images, speech, and time series," *The handbook of brain theory and neural networks*, vol. 3361, no. 10, p. 1995, 1995.

[10] J. Yangqing, "Caffe LeNet-5," https://github.com/BVLC/caffe/tree/master/examples/mnist/, 2014.

[11] X. Zhang, X. Zhou, M. Lin, and J. Sun, "Shufflenet: An extremely efficient convolutional neural network for mobile devices," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2018, pp. 6848–6856.

[12] X. Ding, X. Zhang, N. Ma, J. Han, G. Ding, and J. Sun, "Repvgg: Making vgg-style convnets great again," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2021, pp. 13 733–13 742.

[13] L.-C. Chen, G. Papandreou, I. Kokkinos, K. Murphy, and A. L. Yuille, "Deeplab: Semantic image segmentation with deep convolutional nets, atrous convolution, and fully connected crfs," *IEEE transactions on pattern analysis and machine intelligence*, vol. 40, no. 4, pp. 834–848, 2017.

[14] F. Chollet, "Xception: Deep learning with depthwise separable convolutions," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2017, pp. 1251–1258.

[15] J. Dai, H. Qi, Y. Xiong, Y. Li, G. Zhang, H. Hu, and Y. Wei, "Deformable convolutional networks," in *Proceedings of the IEEE international conference on computer vision*, 2017, pp. 764–773.

[16] Z. Liu, H. Mao, C.-Y. Wu, C. Feichtenhofer, T. Darrell, and S. Xie, "A convnet for the 2020s," *arXiv preprint arXiv:2201.03545*, 2022.

[17] A. G. Howard, M. Zhu, B. Chen, D. Kalenichenko, W. Wang, T. Weyand, M. Andreetto, and H. Adam, "Mobilenets: Efficient convolutional neural networks for mobile vision applications," *arXiv preprint arXiv:1704.04861*, 2017.

[18] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," *arXiv preprint arXiv:1409.1556*, 2014.

[19] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, "An image is worth 16x16 words: Transformers for image recognition at scale," in *9th International Conference on Learning Representations, ICLR 2021, Virtual Event, Austria, May 3-7, 2021*. OpenReview.net, 2021. [Online]. Available: https://openreview.net/forum?id=YicbFdNTTy

[20] Z. Liu, Y. Lin, Y. Cao, H. Hu, Y. Wei, Z. Zhang, S. Lin, and B. Guo, "Swin transformer: Hierarchical vision transformer using shifted windows," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2021, pp. 10 012–10 022.

[21] H. Touvron, M. Cord, M. Douze, F. Massa, A. Sablayrolles, and H. Jégou, "Training data-efficient image transformers & distillation through attention," in *International Conference on Machine Learning*. PMLR, 2021, pp. 10 347–10 357.

[22] W. Wang, E. Xie, X. Li, D.-P. Fan, K. Song, D. Liang, T. Lu, P. Luo, and L. Shao, "Pyramid vision transformer: A versatile backbone for dense prediction without convolutions," *arXiv preprint arXiv:2102.12122*, 2021.

[23] C. Ge, X. Ding, Z. Tong, L. Yuan, J. Wang, Y. Song, and P. Luo, "Advancing vision transformers with group-mix attention," *arXiv preprint arXiv:2311.15157*, 2023.

[24] A. Srinivas, T.-Y. Lin, N. Parmar, J. Shlens, P. Abbeel, and A. Vaswani, "Bottleneck transformers for visual recognition," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2021, pp. 16 519–16 529.

[25] A. Vaswani, P. Ramachandran, A. Srinivas, N. Parmar, B. Hechtman, and J. Shlens, "Scaling local self-attention for parameter efficient visual backbones," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2021, pp. 12 894–12 904.

[26] P. Ramachandran, N. Parmar, A. Vaswani, I. Bello, A. Levskaya, and J. Shlens, "Stand-alone self-attention in vision models," *arXiv preprint arXiv:1906.05909*, 2019.

[27] R. Girdhar, A. El-Nouby, Z. Liu, M. Singh, K. V. Alwala, A. Joulin, and I. Misra, "Imagebind: One embedding space to bind them all," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023, pp. 15 180–15 190.

[28] Y. Zhang, K. Gong, K. Zhang, H. Li, Y. Qiao, W. Ouyang, and X. Yue, "Meta-transformer: A unified framework for multimodal learning," *arXiv preprint arXiv:2307.10802*, 2023.

[29] J. Han, K. Gong, Y. Zhang, J. Wang, K. Zhang, D. Lin, Y. Qiao, P. Gao, and X. Yue, "Onellm: One framework to align all modalities with language," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024, pp. 26 584–26 595.

[30] Y. Gong, Y.-A. Chung, and J. Glass, "Ast: Audio spectrogram transformer," *arXiv preprint arXiv:2104.01778*, 2021.

[31] H. Zhao, L. Jiang, J. Jia, P. Torr, and V. Koltun, "Point transformer," in *ICCV*, 2021.

[32] Y. Li, C.-Y. Wu, H. Fan, K. Mangalam, B. Xiong, J. Malik, and C. Feichtenhofer, "Mvitv2: Improved multiscale vision transformers for classification and detection," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022, pp. 4804–4814.

[33] G. Hinton, "How to represent part-whole hierarchies in a neural network," *arXiv preprint arXiv:2102.12627*, 2021.

[34] X. Zhu, D. Cheng, Z. Zhang, S. Lin, and J. Dai, "An empirical study of spatial attention mechanisms in deep networks," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2019, pp. 6688–6697.

[35] Q. Han, Z. Fan, Q. Dai, L. Sun, M.-M. Cheng, J. Liu, and J. Wang, "Demystifying local vision transformer: Sparse connectivity, weight sharing, and dynamic weight," *arXiv preprint arXiv:2106.04263*, 2021.

[36] F. Wu, A. Fan, A. Baevski, Y. N. Dauphin, and M. Auli, "Pay less attention with lightweight and dynamic convolutions," *arXiv preprint arXiv:1901.10430*, 2019.

[37] J.-B. Cordonnier, A. Loukas, and M. Jaggi, "On the relationship between self-attention and convolutional layers," *arXiv preprint arXiv:1911.03584*, 2019.

[38] L. Xu, J. S. Ren, C. Liu, and J. Jia, "Deep convolutional neural network for image deconvolution," *Advances in neural information processing systems*, vol. 27, 2014.

[39] C. Peng, X. Zhang, G. Yu, G. Luo, and J. Sun, "Large kernel matters–improve semantic segmentation by global convolutional network," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2017, pp. 4353–4361.

[40] S. Liu, T. Chen, X. Chen, X. Chen, Q. Xiao, B. Wu, T. Kärkkäinen, M. Pechenizkiy, D. Mocanu, and Z. Wang, "More convnets in the 2020s: Scaling up kernels beyond 51x51 using sparsity," *arXiv preprint arXiv:2207.03620*, 2022.

[41] Z. Liu, H. Hu, Y. Lin, Z. Yao, Z. Xie, Y. Wei, J. Ning, Y. Cao, Z. Zhang, L. Dong *et al.*, "Swin transformer v2: Scaling up capacity and resolution," *arXiv preprint arXiv:2111.09883*, 2021.

[42] W. Luo, Y. Li, R. Urtasun, and R. S. Zemel, "Understanding the effective receptive field in deep convolutional neural networks,"

in *Advances in Neural Information Processing Systems 29: Annual Conference on Neural Information Processing Systems 2016, December 5-10, 2016, Barcelona, Spain*, D. D. Lee, M. Sugiyama, U. von Luxburg, I. Guyon, and R. Garnett, Eds., 2016, pp. 4898–4906. [Online]. Available: https://proceedings.neurips.cc/paper/2016/hash/c8067ad1937f728f51288b3eb986afaa-Abstract.html

[43] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "Imagenet: A large-scale hierarchical image database," in *Computer Vision and Pattern Recognition, 2009. CVPR 2009. IEEE Conference on*. IEEE, 2009, pp. 248–255.

[44] J. F. Gemmeke, D. P. Ellis, D. Freedman, A. Jansen, W. Lawrence, R. C. Moore, M. Plakal, and M. Ritter, "Audio set: An ontology and human-labeled dataset for audio events," in *2017 IEEE international conference on acoustics, speech and signal processing (ICASSP)*. IEEE, 2017, pp. 776–780.

[45] M. A. Uy, Q.-H. Pham, B.-S. Hua, D. T. Nguyen, and S.-K. Yeung, "Revisiting point cloud classification: A new benchmark dataset and classification model on real-world data," in *International Conference on Computer Vision (ICCV)*, 2019.

[46] H. Wu, H. Zhou, M. Long, and J. Wang, "Interpretable weather forecasting for worldwide stations with a unified deep model," *Nature Machine Intelligence*, pp. 1–10, 2023.

[47] S. Woo, S. Debnath, R. Hu, X. Chen, Z. Liu, I. S. Kweon, and S. Xie, "Convnext v2: Co-designing and scaling convnets with masked autoencoders," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023, pp. 16 133–16 142.

[48] P. K. A. Vasu, J. Gabriel, J. Zhu, O. Tuzel, and A. Ranjan, "Fastvit: A fast hybrid vision transformer using structural reparameterization," *arXiv preprint arXiv:2303.14189*, 2023.

[49] Z. Liu, H. Hu, Y. Lin, Z. Yao, Z. Xie, Y. Wei, J. Ning, Y. Cao, Z. Zhang, L. Dong *et al.*, "Swin transformer v2: Scaling up capacity and resolution," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2022, pp. 12 009–12 019.

[50] H. Touvron, M. Cord, and H. Jégou, "Deit iii: Revenge of the vit," in *European Conference on Computer Vision*. Springer, 2022, pp. 516–533.

[51] S. Tuli, I. Dasgupta, E. Grant, and T. L. Griffiths, "Are convolutional neural networks or transformers more like human vision?" *arXiv preprint arXiv:2105.07197*, 2021.

[52] bethgelab, "Toolbox of model-vs-human," https://github.com/bethgelab/model-vs-human, 2022.

[53] C. Schuhmann, R. Beaumont, R. Vencu, C. Gordon, R. Wightman, M. Cherti, T. Coombes, A. Katta, C. Mullis, M. Wortsman *et al.*, "Laion-5b: An open large-scale dataset for training next generation image-text models," *Advances in Neural Information Processing Systems*, vol. 35, pp. 25 278–25 294, 2022.

[54] G. Ilharco, M. Wortsman, R. Wightman, C. Gordon, N. Carlini, R. Taori, A. Dave, V. Shankar, H. Namkoong, J. Miller, H. Hajishirzi, A. Farhadi, and L. Schmidt, "Openclip," Jul. 2021, if you use this software, please cite it as below. [Online]. Available: https://doi.org/10.5281/zenodo.5143773

[55] Y. Zhang, X. Ding, K. Gong, Y. Ge, Y. Shan, and X. Yue, "Multimodal pathway: Improve transformers with irrelevant data from other modalities," *arXiv preprint arXiv:2401.14405*, 2024.

[56] C. Jia, Y. Yang, Y. Xia, Y.-T. Chen, Z. Parekh, H. Pham, Q. V. Le, Y. Sung, Z. Li, and T. Duerig, "Scaling up visual and vision-language representation learning with noisy text supervision," *arXiv preprint arXiv:2102.05918*, 2021.

[57] H. Hu, Z. Zhang, Z. Xie, and S. Lin, "Local relation networks for image recognition," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2019, pp. 3464–3473.

[58] W. Yu, M. Luo, P. Zhou, C. Si, Y. Zhou, X. Wang, J. Feng, and S. Yan, "Metaformer is actually what you need for vision," *arXiv preprint arXiv:2111.11418*, 2021.

[59] Y. Rao, W. Zhao, Z. Zhu, J. Lu, and J. Zhou, "Global filter networks for image classification," *arXiv preprint arXiv:2107.00645*, 2021.

[60] Y. Chen, J. Liu, X. Zhang, X. Qi, and J. Jia, "Largekernel3d: Scaling up kernels in 3d sparse cnns," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023, pp. 13 488–13 498.

[61] P. Luo, G. Xiao, X. Gao, and S. Wu, "Lkd-net: Large kernel convolution network for single image dehazing," in *2023 IEEE International Conference on Multimedia and Expo (ICME)*. IEEE, 2023, pp. 1601–1606.

[62] C. Xie, X. Zhang, L. Li, H. Meng, T. Zhang, T. Li, and X. Zhao, "Large kernel distillation network for efficient single image super-resolution," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023, pp. 1283–1292.

[63] J. Hu, L. Shen, and G. Sun, "Squeeze-and-excitation networks," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2018, pp. 7132–7141.

[64] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," in *International Conference on Machine Learning*, 2015, pp. 448–456.

[65] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "Mobilenetv2: Inverted residuals and linear bottlenecks," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2018, pp. 4510–4520.

[66] N. Ma, X. Zhang, H.-T. Zheng, and J. Sun, "Shufflenet v2: Practical guidelines for efficient cnn architecture design," in *Proceedings of the European conference on computer vision (ECCV)*, 2018, pp. 116–131.

[67] A. Veit, M. J. Wilber, and S. Belongie, "Residual networks behave like ensembles of relatively shallow networks," in *Advances in neural information processing systems*, 2016, pp. 550–558.

[68] X. Ding, Y. Guo, G. Ding, and J. Han, "Acnet: Strengthening the kernel skeletons for powerful cnn via asymmetric convolution blocks," in *Proceedings of the IEEE International Conference on Computer Vision*, 2019, pp. 1911–1920.

[69] X. Ding, H. Chen, X. Zhang, J. Han, and G. Ding, "Repmlpnet: Hierarchical vision mlp with re-parameterized locality," *arXiv preprint arXiv:2112.11081*, 2021.

[70] L.-C. Chen, Y. Zhu, G. Papandreou, F. Schroff, and H. Adam, "Encoder-decoder with atrous separable convolution for semantic image segmentation," in *Proceedings of the European conference on computer vision (ECCV)*, 2018, pp. 801–818.

[71] M. Cordts, M. Omran, S. Ramos, T. Rehfeld, M. Enzweiler, R. Benenson, U. Franke, S. Roth, and B. Schiele, "The cityscapes dataset for semantic urban scene understanding," in *2016 IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2016, Las Vegas, NV, USA, June 27-30, 2016*. IEEE Computer Society, 2016, pp. 3213–3223. [Online]. Available: https://doi.org/10.1109/CVPR.2016.350

[72] M. Contributors, "MMSegmentation: Openmmlab semantic segmentation toolbox and benchmark," https://github.com/open-mmlab/mmsegmentation, 2020.

[73] P. Shaw, J. Uszkoreit, and A. Vaswani, "Self-attention with relative position representations," *arXiv preprint arXiv:1803.02155*, 2018.

[74] I. Bello, B. Zoph, A. Vaswani, J. Shlens, and Q. V. Le, "Attention augmented convolutional networks," in *Proceedings of the IEEE/CVF international conference on computer vision*, 2019, pp. 3286–3295.

[75] O. S. Kayhan and J. C. v. Gemert, "On translation invariance in cnns: Convolutional layers can exploit absolute spatial location," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2020, pp. 14 274–14 285.

[76] J. Long, E. Shelhamer, and T. Darrell, "Fully convolutional networks for semantic segmentation," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2015, pp. 3431–3440.

[77] F. Yu, V. Koltun, and T. Funkhouser, "Dilated residual networks," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2017, pp. 472–480.

[78] J. Wang, K. Sun, T. Cheng, B. Jiang, C. Deng, Y. Zhao, D. Liu, Y. Mu, M. Tan, X. Wang *et al.*, "Deep high-resolution representation learning for visual recognition," *IEEE transactions on pattern analysis and machine intelligence*, 2020.

[79] F. Yu and V. Koltun, "Multi-scale context aggregation by dilated convolutions," *arXiv preprint arXiv:1511.07122*, 2015.

[80] R. Geirhos, P. Rubisch, C. Michaelis, M. Bethge, F. A. Wichmann, and W. Brendel, "Imagenet-trained cnns are biased towards texture; increasing shape bias improves accuracy and robustness," *arXiv preprint arXiv:1811.12231*, 2018.

[81] W. Brendel and M. Bethge, "Approximating cnns with bag-of-local-features models works surprisingly well on imagenet," *arXiv preprint arXiv:1904.00760*, 2019.

[82] J. L. Ba, J. R. Kiros, and G. E. Hinton, "Layer normalization," *arXiv preprint arXiv:1607.06450*, 2016.

[83] T. Xiao, Y. Liu, B. Zhou, Y. Jiang, and J. Sun, "Unified perceptual parsing for scene understanding," in *Proceedings of the European Conference on Computer Vision (ECCV)*, 2018, pp. 418–434.

[84] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in neural information processing systems*, 2017, pp. 5998–6008.

[85] W. Han, X. Dong, Y. Zhang, D. Crandall, C.-Z. Xu, and J. Shen, "Asymmetric convolution: An efficient and generalized method to fuse feature maps in multiple vision tasks," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2024.

[86] W. Wang, E. Xie, X. Li, D.-P. Fan, K. Song, D. Liang, T. Lu, P. Luo, and L. Shao, "Pvt v2: Improved baselines with pyramid vision transformer," *Computational Visual Media*, vol. 8, no. 3, pp. 415–424, 2022.

[87] Z. Dai, H. Liu, Q. V. Le, and M. Tan, "Coatnet: Marrying convolution and attention for all data sizes," *Advances in neural information processing systems*, vol. 34, pp. 3965–3977, 2021.

[88] W. Wang, J. Dai, Z. Chen, Z. Huang, Z. Li, X. Zhu, X. Hu, T. Lu, L. Lu, H. Li *et al.*, "Internimage: Exploring large-scale vision foundation models with deformable convolutions," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023, pp. 14 408–14 419.

[89] Y. Rao, W. Zhao, Y. Tang, J. Zhou, S. N. Lim, and J. Lu, "Hornet: Efficient high-order spatial interactions with recursive gated convolutions," *Advances in Neural Information Processing Systems*, vol. 35, pp. 10 353–10 366, 2022.

[90] Q. Kong, Y. Cao, T. Iqbal, Y. Wang, W. Wang, and M. D. Plumbley, "Panns: Large-scale pretrained audio neural networks for audio pattern recognition," *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 28, pp. 2880–2894, 2020.

[91] Y. Gong, Y.-A. Chung, and J. Glass, "Psla: Improving audio tagging with pretraining, sampling, labeling, and aggregation," *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 29, pp. 3292–3306, 2021.

[92] Y. Gong, C.-I. Lai, Y.-A. Chung, and J. Glass, "Ssast: Self-supervised audio spectrogram transformer," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 36, no. 10, 2022, pp. 10 699–10 709.

[93] P.-Y. Huang, H. Xu, J. Li, A. Baevski, M. Auli, W. Galuba, F. Metze, and C. Feichtenhofer, "Masked autoencoders that listen," *Advances in Neural Information Processing Systems*, vol. 35, pp. 28 708–28 720, 2022.

[94] C. Feichtenhofer, H. Fan, J. Malik, and K. He, "Slowfast networks for video recognition," in *Proceedings of the IEEE/CVF international conference on computer vision*, 2019, pp. 6202–6211.

[95] G. Bertasius, H. Wang, and L. Torresani, "Is space-time attention all you need for video understanding?" in *ICML*, vol. 2, no. 3, 2021, p. 4.

[96] C. R. Qi, H. Su, K. Mo, and L. J. Guibas, "Pointnet: Deep learning on point sets for 3d classification and segmentation," in *CVPR*, 2017.

[97] C. R. Qi, L. Yi, H. Su, and L. J. Guibas, "Pointnet++: Deep hierarchical feature learning on point sets in a metric space," in *NeurIPS*, 2017.

[98] W. Wu, Z. Qi, and L. Fuxin, "Pointconv: Deep convolutional networks on 3d point clouds," in *CVPR*, 2019.

[99] H. Thomas, C. R. Qi, J.-E. Deschaud, B. Marcotegui, F. Goulette, and L. J. Guibas, "Kpconv: Flexible and deformable convolution for point clouds," in *ICCV*, 2019.

[100] Y. Wang, Y. Sun, Z. Liu, S. E. Sarma, M. M. Bronstein, and J. M. Solomon, "Dynamic graph cnn for learning on point clouds," *TOG*, 2019.

[101] M. Liu, R. Shi, K. Kuang, Y. Zhu, X. Li, S. Han, H. Cai, F. Porikli, and H. Su, "Openshape: Scaling up 3d shape representation towards open-world understanding," *arXiv preprint arXiv:2305.10764*, 2023.

[102] K. He, G. Gkioxari, P. Dollár, and R. Girshick, "Mask r-cnn," in *Proceedings of the IEEE international conference on computer vision*, 2017, pp. 2961–2969.

[103] Z. Cai and N. Vasconcelos, "Cascade r-cnn: High quality object detection and instance segmentation," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2019.

[104] K. Chen, J. Wang, J. Pang, Y. Cao, Y. Xiong, X. Li, S. Sun, W. Feng, Z. Liu, J. Xu, Z. Zhang, D. Cheng, C. Zhu, T. Cheng, Q. Zhao, B. Li, X. Lu, R. Zhu, Y. Wu, J. Dai, J. Wang, J. Shi, W. Ouyang, C. C. Loy, and D. Lin, "MMDetection: Open mmlab detection toolbox and benchmark," *arXiv preprint arXiv:1906.07155*, 2019.

[105] B. Zhou, H. Zhao, X. Puig, T. Xiao, S. Fidler, A. Barriuso, and A. Torralba, "Semantic understanding of scenes through the ade20k dataset," *International Journal of Computer Vision*, vol. 127, no. 3, pp. 302–321, 2019.

[106] R. J. Hyndman and G. Athanasopoulos, "Forecasting: Principles and practice. otexts; 2014," *Online at http://otexts. org/fpp*, 2017.

[107] S. J. Taylor and B. Letham, "Forecasting at scale," *The American Statistician*, vol. 72, no. 1, pp. 37–45, 2018.

[108] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "Lightgbm: A highly efficient gradient boosting decision tree," *Advances in neural information processing systems*, vol. 30, 2017.

[109] H. Hersbach, B. Bell, P. Berrisford, S. Hirahara, A. Horányi, J. Muñoz-Sabater, J. Nicolas, C. Peubey, R. Radu, D. Schepers *et al.*, "The era5 global reanalysis, qj roy. meteor. soc., 146, 1999–2049," 2020.

[110] D. Salinas, V. Flunkert, J. Gasthaus, and T. Januschowski, "Deepar: Probabilistic forecasting with autoregressive recurrent networks," *International Journal of Forecasting*, vol. 36, no. 3, pp. 1181–1191, 2020.

[111] B. N. Oreshkin, D. Carpov, N. Chapados, and Y. Bengio, "N-beats: Neural basis expansion analysis for interpretable time series forecasting," in *International Conference on Learning Representations*, 2019.

[112] D. Cao, Y. Wang, J. Duan, C. Zhang, X. Zhu, C. Huang, Y. Tong, B. Xu, J. Bai, J. Tong *et al.*, "Spectral temporal graph neural network for multivariate time-series forecasting," *Advances in neural information processing systems*, vol. 33, pp. 17 766–17 778, 2020.

[113] S. Liu, H. Yu, C. Liao, J. Li, W. Lin, A. X. Liu, and S. Dustdar, "Pyraformer: Low-complexity pyramidal attention for long-range time series modeling and forecasting," in *International conference on learning representations*, 2021.

[114] B. Recht, R. Roelofs, L. Schmidt, and V. Shankar, "Do imagenet classifiers generalize to imagenet?" 2019.

[115] D. Hendrycks, K. Zhao, S. Basart, J. Steinhardt, and D. Song, "Natural adversarial examples," in *CVPR*, 2021.

[116] D. Hendrycks, S. Basart, N. Mu, S. Kadavath, F. Wang, E. Dorundo, R. Desai, T. Zhu, S. Parajuli, M. Guo *et al.*, "The many faces of robustness: A critical analysis of out-of-distribution generalization," in *CVPR*, 2021.

[117] H. Wang, S. Ge, Z. Lipton, and E. P. Xing, "Learning robust global representations by penalizing local predictive power," *NeurIPS*, 2019.

[118] A. Barbu, D. Mayo, J. Alverio, W. Luo, C. Wang, D. Gutfreund, J. Tenenbaum, and B. Katz, "Objectnet: A large-scale bias-controlled dataset for pushing the limits of object recognition models," in *NeurIPS*, 2019.

[119] A. Krizhevsky, G. Hinton *et al.*, "Learning multiple layers of features from tiny images," 2009.

[120] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.

[121] L. Fei-Fei, R. Fergus, and P. Perona, "Learning generative visual models from few training examples: An incremental bayesian approach tested on 101 object categories," in *CVPRW*, 2004.

[122] J. Xiao, J. Hays, K. A. Ehinger, A. Oliva, and A. Torralba, "Sun database: Large-scale scene recognition from abbey to zoo," in *CVPR*, 2010.

[123] S. Maji, E. Rahtu, J. Kannala, M. Blaschko, and A. Vedaldi, "Fine-grained visual classification of aircraft," *arXiv preprint arXiv:1306.5151*, 2013.

[124] J. Krause, M. Stark, J. Deng, and L. Fei-Fei, "3d object representations for fine-grained categorization," in *ICCVW*, 2013.

[125] M. Cimpoi, S. Maji, I. Kokkinos, S. Mohamed, , and A. Vedaldi, "Describing textures in the wild," in *CVPR*, 2014.

[126] P. Helber, B. Bischke, A. Dengel, and D. Borth, "Eurosat: A novel dataset and deep learning benchmark for land use and land cover classification," *IEEE J. Sel. Top. Appl. Earth Obs. Remote Sens.*, 2019.

[127] I. J. Goodfellow, D. Erhan, P. L. Carrier, A. Courville, M. Mirza, B. Hamner, W. Cukierski, Y. Tang, D. Thaler, D.-H. Lee *et al.*, "Challenges in representation learning: A report on three machine learning contests," in *ICONIP*, 2013.

[128] M.-E. Nilsback and A. Zisserman, "Automated flower classification over a large number of classes," in *ICVGIP*, 2008.

[129] L. Bossard, M. Guillaumin, and L. Van Gool, "Food-101–mining discriminative components with random forests," in *ECCV*, 2014.

[130] J. Stallkamp, M. Schlipsing, J. Salmen, and C. Igel, "Man vs. computer: Benchmarking machine learning algorithms for traffic sign recognition," *Neural networks*, 2012.

[131] B. S. Veeling, J. Linmans, J. Winkens, T. Cohen, and M. Welling, "Rotation equivariant cnns for digital pathology," in *MICCAI*, 2018.

[132] O. M. Parkhi, A. Vedaldi, A. Zisserman, and C. V. Jawahar, "Cats and dogs," in *CVPR*, 2012.

[133] G. Cheng, J. Han, and X. Lu, "Remote sensing image scene classification: Benchmark and state of the art," *Proceedings of the IEEE*, 2017.

[134] A. Coates, A. Ng, and H. Lee, "An analysis of single-layer networks in unsupervised feature learning," in *AISTAT*, 2011.

[135] M. Everingham, L. Van Gool, C. K. I. Williams, J. Winn, and A. Zisserman, ""the PASCAL Visual Object Classes Challenge 2007 (VOC2007) Results," "http://www.pascal-network.org/challenges/VOC/voc2007/workshop/index.html", 2007.

[136] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark *et al.*, "Learning transferable visual models from natural language supervision," in *International conference on machine learning*. PMLR, 2021, pp. 8748–8763.

[137] Y. Li, H. Fan, R. Hu, C. Feichtenhofer, and K. He, "Scaling language-image pre-training via masking," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023, pp. 23 390–23 400.

[138] Q. Sun, Y. Fang, L. Wu, X. Wang, and Y. Cao, "Eva-clip: Improved training techniques for clip at scale," *arXiv preprint arXiv:2303.15389*, 2023.

[139] D. A. Hudson and C. D. Manning, "Gqa: A new dataset for real-world visual reasoning and compositional question answering," in *CVPR*, 2019.

[140] Y. Goyal, T. Khot, D. Summers-Stay, D. Batra, and D. Parikh, "Making the v in vqa matter: Elevating the role of image understanding in visual question answering," in *CVPR*, 2017, pp. 6904–6913.

[141] K. Marino, M. Rastegari, A. Farhadi, and R. Mottaghi, "Ok-vqa: A visual question answering benchmark requiring external knowledge," in *CVPR*, 2019.

[142] A. Singh, V. Natarajan, M. Shah, Y. Jiang, X. Chen, D. Batra, D. Parikh, and M. Rohrbach, "Towards vqa models that can read," in *CVPR*, 2019, pp. 8317–8326.

[143] P. Lu, S. Mishra, T. Xia, L. Qiu, K.-W. Chang, S.-C. Zhu, O. Tafjord, P. Clark, and A. Kalyan, "Learn to explain: Multimodal reasoning via thought chains for science question answering," *NeurIPS*, 2022.

[144] D. Gurari, Q. Li, A. J. Stangl, A. Guo, C. Lin, K. Grauman, J. Luo, and J. P. Bigham, "Vizwiz grand challenge: Answering visual questions from blind people," in *CVPR*, 2018, pp. 3608–3617.

[145] H. Agrawal, K. Desai, Y. Wang, X. Chen, R. Jain, M. Johnson, D. Batra, D. Parikh, S. Lee, and P. Anderson, "nocaps: novel object captioning at scale," in *ICCV*, 2019.

[146] B. A. Plummer, L. Wang, C. M. Cervantes, J. C. Caicedo, J. Hockenmaier, and S. Lazebnik, "Flickr30k entities: Collecting region-to-phrase correspondences for richer image-to-sentence models," in *ICCV*, 2015, pp. 2641–2649.

[147] C. Fu, P. Chen, Y. Shen, Y. Qin, M. Zhang, X. Lin, Z. Qiu, W. Lin, J. Yang, X. Zheng *et al.*, "Mme: A comprehensive evaluation benchmark for multimodal large language models," *arXiv preprint arXiv:2306.13394*, 2023.

[148] Y. Liu, H. Duan, Y. Zhang, B. Li, S. Zhang, W. Zhao, Y. Yuan, J. Wang, C. He, Z. Liu *et al.*, "Mmbench: Is your multi-modal model an all-around player?" *arXiv preprint arXiv:2307.06281*, 2023.

[149] W. Yu, Z. Yang, L. Li, J. Wang, K. Lin, Z. Liu, X. Wang, and L. Wang, "Mm-vet: Evaluating large multimodal models for integrated capabilities," *arXiv preprint arXiv:2308.02490*, 2023.

[150] B. Li, R. Wang, G. Wang, Y. Ge, Y. Ge, and Y. Shan, "Seed-bench: Benchmarking multimodal llms with generative comprehension," *arXiv preprint arXiv:2307.16125*, 2023.

[151] J.-B. Alayrac, J. Donahue, P. Luc, A. Miech, I. Barr, Y. Hasson, K. Lenc, A. Mensch, K. Millican, M. Reynolds *et al.*, "Flamingo: a visual language model for few-shot learning," *NeurIPS*, vol. 35, pp. 23 716–23 736, 2022.

[152] J. Li, D. Li, S. Savarese, and S. Hoi, "Blip-2: Bootstrapping language-image pre-training with frozen image encoders and large language models," *arXiv preprint arXiv:2301.12597*, 2023.

[153] W. Dai, J. Li, D. Li, A. M. H. Tiong, J. Zhao, W. Wang, B. Li, P. Fung, and S. Hoi, "Instructblip: Towards general-purpose vision-language models with instruction tuning," 2023.

[154] H. Laurençon, L. Saulnier, L. Tronchon, S. Bekman, A. Singh, A. Lozhkov, T. Wang, S. Karamcheti, A. M. Rush, D. Kiela *et al.*, "Obelisc: An open web-scale filtered dataset of interleaved image-text documents," *arXiv preprint arXiv:2306.16527*, 2023.

[155] J. Bai, S. Bai, S. Yang, S. Wang, S. Tan, P. Wang, J. Lin, C. Zhou, and J. Zhou, "Qwen-vl: A frontier large vision-language model with versatile abilities," *arXiv preprint arXiv:2308.12966*, 2023.

[156] H. Liu, C. Li, Y. Li, and Y. J. Lee, "Improved baselines with visual instruction tuning," 2023.

[157] J. Han, R. Zhang, W. Shao, P. Gao, P. Xu, H. Xiao, K. Zhang, C. Liu, S. Wen, Z. Guo *et al.*, "Imagebind-llm: Multi-modality instruction tuning," *arXiv preprint arXiv:2309.03905*, 2023.

[158] S. Moon, A. Madotto, Z. Lin, T. Nagarajan, M. Smith, S. Jain, C.-F. Yeh, P. Murugesan, P. Heidari, Y. Liu *et al.*, "Anymal: An efficient and scalable any-modality augmented language model," *arXiv preprint arXiv:2309.16058*, 2023.

[159] P. Warden, "Speech commands: A dataset for limited-vocabulary speech recognition," *arXiv preprint arXiv:1804.03209*, 2018.

[160] W. Kay, J. Carreira, K. Simonyan, B. Zhang, C. Hillier, S. Vijayanarasimhan, F. Viola, T. Green, T. Back, P. Natsev *et al.*, "The kinetics human action video dataset," *arXiv preprint arXiv:1705.06950*, 2017.

[161] Z. Wu, S. Song, A. Khosla, F. Yu, L. Zhang, X. Tang, and J. Xiao, "3d shapenets: A deep representation for volumetric shapes," in *CVPR*, 2015.

[162] J. Pont-Tuset, F. Perazzi, S. Caelles, P. Arbeláez, A. Sorkine-Hornung, and L. Van Gool, "The 2017 davis challenge on video object segmentation," *arXiv preprint arXiv:1704.00675*, 2017.

[163] A. Khoreva, A. Rohrbach, and B. Schiele, "Video object segmentation with language referring expressions," in *Computer Vision–ACCV 2018: 14th Asian Conference on Computer Vision, Perth, Australia, December 2–6, 2018, Revised Selected Papers, Part IV 14*. Springer, 2019, pp. 123–141.

[164] S. Seo, J.-Y. Lee, and B. Han, "Urvos: Unified referring video object segmentation network with a large-scale benchmark," in *Computer Vision–ECCV 2020: 16th European Conference, Glasgow, UK, August 23–28, 2020, Proceedings, Part XV 16*. Springer, 2020, pp. 208–223.

[165] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," *arXiv preprint arXiv:1711.05101*, 2017.

[166] B. Zhou, H. Zhao, X. Puig, S. Fidler, A. Barriuso, and A. Torralba, "Scene parsing through ade20k dataset," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2017, pp. 633–641.

APPENDIX A: GENERAL TRANSFORMATION FROM DIALTED CONVOLUTION TO NON-DILATED LARGE-KERNEL CONVOLUTION

Since *ignoring pixels of the input is equivalent to inserting extra zero entries into the conv kernel, a dilated conv layer with a small kernel can be equivalently converted into a non-dilated layer with a sparse larger kernel*. Let k be the kernel size and r be the dilation rate of the dilated layer, by inserting zero entries, the kernel size of the corresponding non-dilated layer will be $(k-1)r+1$, which is referred to as the *equivalent kernel size* for brevity.

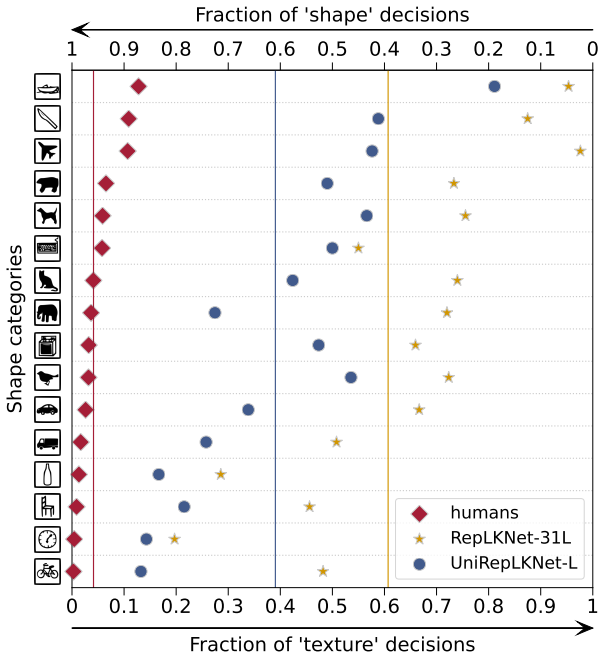

Fig. 8: Shape bias of ImageNet-22K-pretrained UniRepLKNet-L and RepLKNet-31L.

As discussed in the paper, to eliminate the inference costs of the extra dilated conv layers in the Dilated Reparam Block, we propose to equivalently transform the whole block into a single non-dilated conv layer for inference. As discussed before, let k and r be the kernel size and dilation rate, respectively, the transformation from a dilated conv layer's kernel $\mathrm{W} \in \mathcal{R}^{k\times k}$ to a non-dilated layer's kernel $\mathrm{W}' \in \mathcal{R}^{((k-1)r+1)\times((k-1)r+1)}$ can be elegantly realized by a transpose convolution with a stride of r and an identity kernel $\mathrm{I} \in \mathcal{R}^{1\times 1}$, which is scalar 1 but viewed as a kernel tensor. That is

$$\mathrm{W}' = \texttt{conv_transpose2d}(\mathrm{W}, \mathrm{I}, \text{stride} = r)\,. \tag{6}$$

In general cases with multi-channel conv layers, let the input channels, output channels, and number of groups be c_{in}, c_{out}, and g, respectively, we denote the kernel by a 4D tensor whose shape is $c_{\text{out}} \times \frac{c_{\text{in}}}{g} \times k \times k$.

1) For a multi-channel depthwise (DW) layer, the transformation is easily generalized from 2D to 4D - the identity kernel I is viewed as a 4D tensor $\mathrm{I} \in \mathcal{R}^{1\times 1\times 1\times 1}$ and we still follow function 6 to derive the equivalent kernel by transpose convolution.

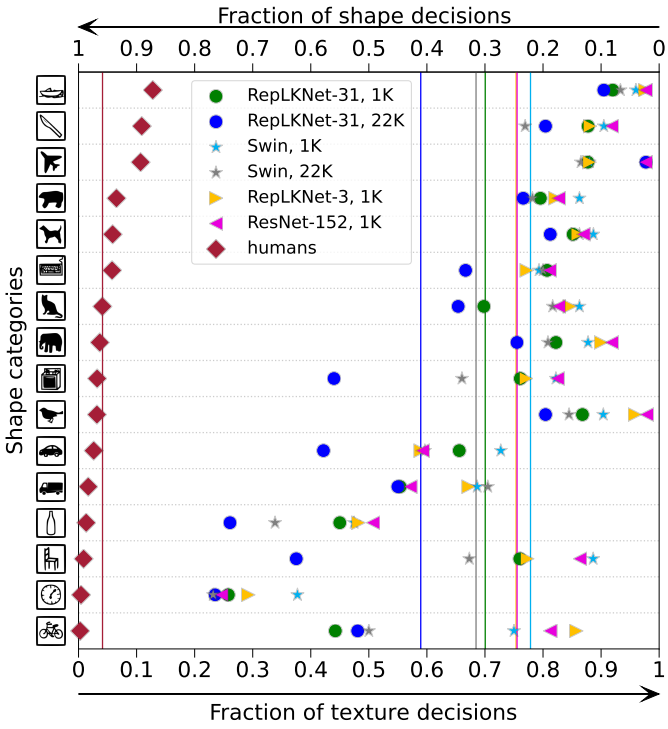

Fig. 9: Shape bias of ImageNet-1K and ImageNet-22K-pretrained RepLKNet-31B and Swin-B. This figure is directly taken from the supplementary material of RepLKNet without any modifications

2) For non-DW cases (*i.e.*, $g < c_{\text{in}}$), the transformation can be seen as splitting the kernel into slices (which can each be seen as a DW kernel), converting the slices respectively, and concatenating the resultant non-dilated slices up. We present the code in pytorch (Fig. 10) and a test case demonstrating the equivalency (Fig. 11).

APPENDIX B: TRAINING CONFIGURATIONS

We present the detailed training configurations for image classification, object detection, and semantic segmentation. We have publicly released a reproducible training script and trained weights for every model on GitHub.

ImageNet image classification. The training configurations for the ImageNet-1K-only results shown in Section 4 are presented in Table 16. These configurations are similar to common practices. For the experiments in Section 3, we use the same configurations, except that the training epochs are set to 100 and the drop path rate is set to 0.1. For the models pretrained with ImageNet-22K and then finetuned on ImageNet-22K, the configurations are shown in Table 16. Note that we follow the configurations adopted by ConvNeXt for a fair comparison with ConvNeXt-S/B, and the configurations used by InternImage for a fair comparison with InternImage-L/XL (the results with ImageNet-22K-pretrained InternImage-S/B were not reported).

COCO object detection. For fair comparisons, we follow common practices [16], [41] to initialize the backbone with pretrained weights and train the models using a 3× (36 epochs) schedule by default. The shorter side is resized to 480−800 pixels, while the longer side does not exceed

```python
import torch
import torch.nn as nn
import torch.nn.functional as F

def convert_dilated_to_nondilated(kernel, dilate_rate):
   identity_kernel = torch.ones((1, 1, 1, 1))
   if kernel.size(1) == 1:
      # This is a DW kernel
      dilated = F.conv_transpose2d(kernel, identity_kernel, stride=dilate_rate)
      return dilated
   else:
      # This is a dense or group-wise (but not DW) kernel
      slices = []
      for i in range(kernel.size(1)):
         dilated = F.conv_transpose2d(kernel[:,i:i+1,:,:], identity_kernel, stride=dilate_rate)
         slices.append(dilated)
      return torch.cat(slices, dim=1)
```

Fig. 10: Pytorch code to convert a dilated conv layer's small kernel to a non-dilated layer's larger sparse kernel.

```python
def test_equivalency(in_channels, out_channels, groups, large_kernel_size, small_conv_r,
    small_conv_k):
   equivalent_kernel_size = small_conv_r * (small_conv_k - 1) + 1
   large_conv = nn.Conv2d(in_channels, out_channels, kernel_size=large_kernel_size,
                     padding=large_kernel_size // 2, groups=groups, bias=False)
   dilated_conv = nn.Conv2d(in_channels, out_channels, kernel_size=small_conv_k,
   padding=equivalent_kernel_size // 2,
                      dilation=small_conv_r, groups=groups, bias=False)
   H, W = 19, 19
   x = torch.rand(2, in_channels, H, W)
   origin_y = large_conv(x) + dilated_conv(x)
   equivalent_kernel = convert_dilated_to_nondilated(dilated_conv.weight.data, small_conv_r)
   rows_to_pad = large_kernel_size // 2 - equivalent_kernel_size // 2
   merged_kernel = large_conv.weight.data + F.pad(equivalent_kernel, [rows_to_pad] * 4)
   equivalent_y = F.conv2d(x, merged_kernel, bias=None, padding=large_kernel_size // 2, groups=
       groups)
   print('relative error:', (equivalent_y - origin_y).abs().sum() / origin_y.abs().sum())

test_equivalency(in_channels=4, out_channels=4, groups=1,
   large_kernel_size=13, small_conv_r=3, small_conv_k=3)
```

Fig. 11: A test case demonstrating the equivalency of the transformation.

1,333 pixels. All the models are trained with a batch size of 16 and AdamW [165] optimizer with an initial learning rate of 1×10^{-4}. We have publicly released the training configuration files used in the MMDetection framework and trained weights.

ADE20K semantic segmentation. We evaluate UniRepLKNet models on the ADE20K dataset [166], and initialize them with the pre-trained classification weights. The learning rate is initialized with 1×10^{-4} and decayed with the polynomial decay schedule with a power of 1.0. Following previous methods [16], [41], the crop size is set to 512 for the ImageNet-1K-pretrained models, and 640 for ImageNet-22K-pretrained models. All segmentation models are trained with a batch size of 16 for 160k iterations. We have publicly released the training configuration files used in the MMSegmentation framework and trained weights.

APPENDIX C: SHAPE BIAS

A higher shape bias means the model makes predictions based more on the shape of objects rather than the textures, *i.e.*, the model behaves more similarly to humans. Therefore, a model with a higher shape bias may transfer better to downstream tasks. UniRepLKNet demonstrates significantly higher shape bias than existing ConvNets and ViTs. Concretely, we test the shape bias of ImageNet-22K-pretrained UniRepLKNet-L and RepLKNet-L with the *modelvshuman* toolbox [9]. Fig. 8 shows a significantly higher shape bias of UniRepLKNet - UniRepLKNet makes 20% more decisions based on the overall shapes of objects. This improvement is particularly remarkable since RepLKNet is already known to have a high shape bias (Fig. 9 is directly taken from the supplementary material of the RepLKNet paper without any modifications).

5.1 Appendix D: Training Memory Footprint

The extra parallel dilated branches in Dilated Reparam Block consume more training resources, which is acceptable considering the performance improvements. We present the

9. https://github.com/bethgelab/model-vs-human

TABLE 16: **Detailed training configurations of ImageNet-1K-only models.** Apart from the configurations shown in the table, we use random left-right flipping, random resized crop, color jitter of 0.4, Auto-augment, and no repeated augmentation for every model.

settings	UniRepLKNet-A	UniRepLKNet-F	UniRepLKNet-P	UniRepLKNet-N	UniRepLKNet-T	UniRepLKNet-S
input scale	224	224	224	224	224	224
batch size	4096	4096	4096	4096	4096	4096
optimizer	AdamW	AdamW	AdamW	AdamW	AdamW	AdamW
LR	4×10^{-3}	4×10^{-3}	4×10^{-3}	4×10^{-3}	4×10^{-3}	4×10^{-3}
LR schedule	cosine	cosine	cosine	cosine	cosine	cosine
weight decay	0.05	0.05	0.05	0.05	0.05	0.05
warmup epochs	5	5	5	5	5	5
epochs	300	300	300	300	300	300
mixup alpha	0.3	0.3	0.3	0.5	0.8	0.8
cutmix alpha	0.3	0.3	0.3	0.5	1.0	1.0
erasing prob.	0.25	0.25	0.25	0.25	0.25	0.25
label smoothing ε	0.1	0.1	0.1	0.1	0.1	0.1
drop path rate	0.0	0.0	0.1	0.1	0.2	0.4

TABLE 17: **Detailed training configurations of models pretrained with ImageNet-22K (IN-22K pt) and then finetuned on ImageNet-1K (IN-1K ft).** Apart from the configurations shown in the table, we use random left-right flipping, random resized crop, color jitter of 0.4, Auto-augment, and no repeated augmentation for every model.

settings	UniRepLKNet-S		UniRepLKNet-B		UniRepLKNet-L		UniRepLKNet-XL	
	IN-22K pt	IN-1K ft	IN-22K pt	IN-1K ft	IN-22K pt	IN-1K ft	IN-22K pt	IN-1K ft
input scale	224	384	224	384	192	384	192	384
batch size	4096	512	4096	512	4096	512	4096	512
optimizer	AdamW	AdamW	AdamW	AdamW	AdamW	AdamW	AdamW	AdamW
LR	4×10^{-3}	5×10^{-5}	4×10^{-3}	5×10^{-5}	4×10^{-3}	5×10^{-5}	4×10^{-3}	5×10^{-5}
LR schedule	cosine	cosine	cosine	cosine	cosine	cosine	cosine	cosine
weight decay	0.05	1×10^{-8}	0.05	1×10^{-8}	0.05	1×10^{-8}	0.05	1×10^{-8}
warmup epochs	5	0	5	0	5	0	5	0
epochs	90	30	90	30	90	20	90	20
mixup alpha	0.8	0.0	0.8	0.0	0.8	0.0	0.8	0.0
cutmix alpha	1.0	0.0	1.0	0.0	1.0	0.0	1.0	0.0
erasing prob.	0.25	0.25	0.25	0.25	0.25	0.25	0.25	0.25
label smoothing	0.1	0.1	0.1	0.1	0.1	0.3	0.1	0.3
drop path rate	0.1	0.2	0.1	0.2	0.1	0.3	0.2	0.3

TABLE 18: Training costs.

	Peak memory	Training throughput
Dilated Reparam Block	24.6GB	6642 images/s
Single large-kernel conv layer	20.8GB	9675 images/s

peak GPU memory footprint and training speed in Table 18. With a bigger model and bigger data, we may trade the performance for higher training speed and lower memory consumption by replacing the Dilated Reparam Block with a single large-kernel conv layer followed by Batch Normalization layer. We test the peak memory footprint and actual training throughput while training UniRepLKNet-S with 224×224 inputs and a batch size of 4096 on a node with eight A100 GPUs. Note that such results are significantly influenced by the hardware environment and specific implementation; thus, they should be considered as references only.